

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**
- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```


i=0

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**
- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```


i=0

- Wenn **i==len(liste)-1** gib liste aus

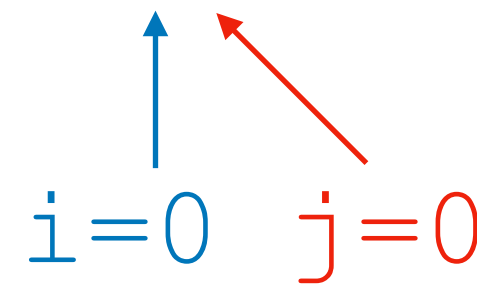
Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**
- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```



- Wenn **i==len(liste)-1** gib liste aus
- Wenn **i<len(liste)-1**
 - Für alle **Indizes j** **j>=i**:

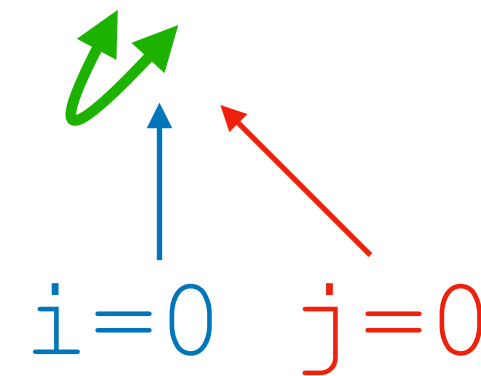
Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**
- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```



- Wenn `i==len(liste)-1` gib liste aus
- Wenn `i<len(liste)-1`
 - Für alle **Indizes** `j>=i`:
 1. **vertausche** `liste[i]` **und** `liste[j]`

Backtracking
Schritt 1

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```

↑
i=1

- Wenn **i==len(liste)-1** gib liste aus
 - Wenn **i<len(liste)-1**
 - Für alle **Indizes j** **j>=i**:
 - Schritt 1** 1. **vertausche** liste[i] **und** liste[j]
 - Schritt 2** 2. verfare **rekursiv** mit **i+=1**

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**
- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```


i=0

- Wenn **i==len(liste)-1** gib liste aus

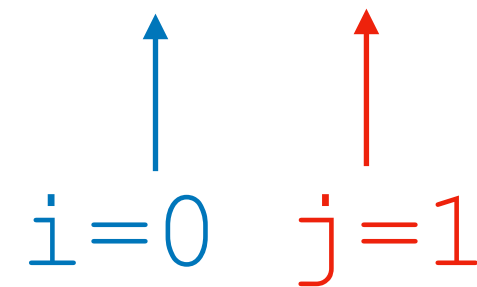
Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**
- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```


i=0 j=1

- Wenn **i==len(liste)-1** gib liste aus
- Wenn **i<len(liste)-1**
 - Für alle **Indizes j** **j>=i**:

Permutationen einer Liste

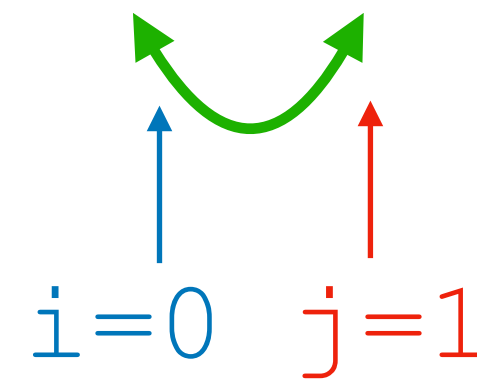
- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```



- Wenn `i==len(liste)-1` gib liste aus
- Wenn `i<len(liste)-1`
 - Für alle **Indizes** `j>=i`:
 1. **vertausche** `liste[i]` **und** `liste[j]`

Backtracking
Schritt 1

Permutationen einer Liste

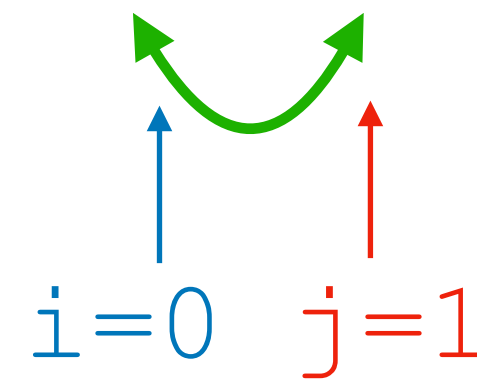
- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[2, 3, 1, 17, 5]
```



- Backtracking
Schritt 1

 - Wenn `i==len(liste)-1` gib liste aus
 - Wenn `i<len(liste)-1`
 - Für alle **Indizes** `j>=i`:
 1. **vertausche** `liste[i]` **und** `liste[j]`

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[2, 3, 1, 17, 5]
```

↑
i=1

- Wenn **i==len(liste)-1** gib liste aus
 - Wenn **i<len(liste)-1**
 - Für alle **Indizes j** **j>=i**:
 - Schritt 1** 1. **vertausche** liste[i] **und** liste[j]
 - Schritt 2** 2. verfare **rekursiv** mit **i+=1**

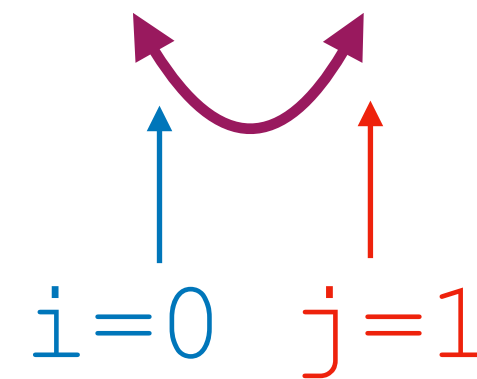
Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**
- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[2, 3, 1, 17, 5]
```



- Wenn `i==len(liste)-1` gib liste aus
- Wenn `i<len(liste)-1`
 - Für alle **Indizes** `j>=i`:
 - Backtracking**
 - Schritt 1** 1. **vertausche** `liste[i]` **und** `liste[j]`
 - Schritt 2** 2. verfare **rekursiv** mit `i+=1`
 - Schritt 3** 3. mache Vertauschung **rückgängig**

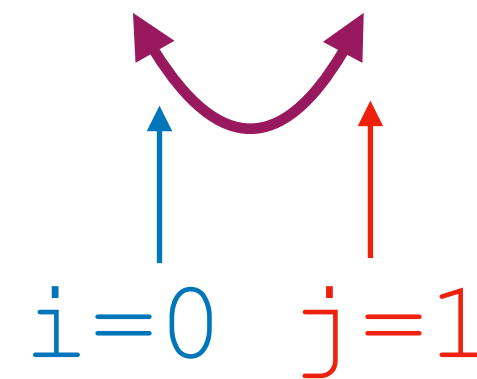
Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**
- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```



- Wenn `i==len(liste)-1` gib liste aus
- Wenn `i<len(liste)-1`
 - Für alle **Indizes** `j>=i`:
 - Backtracking**
 - Schritt 1** 1. **vertausche** `liste[i]` **und** `liste[j]`
 - Schritt 2** 2. verfare **rekursiv** mit `i+=1`
 - Schritt 3** 3. mache Vertauschung **rückgängig**

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```

i=0 j=2

- Wenn **i==len(liste)-1** gib liste aus
- Wenn **i<len(liste)-1**
 - Für alle **Indizes j >= i**:
 - Backtracking**
 - Schritt 1** 1. **vertausche** liste[i] **und** liste[j]
 - Schritt 2** 2. verfare **rekursiv** mit **i+=1**
 - Schritt 3** 3. mache Vertauschung **rückgängig**

Permutationen einer Liste

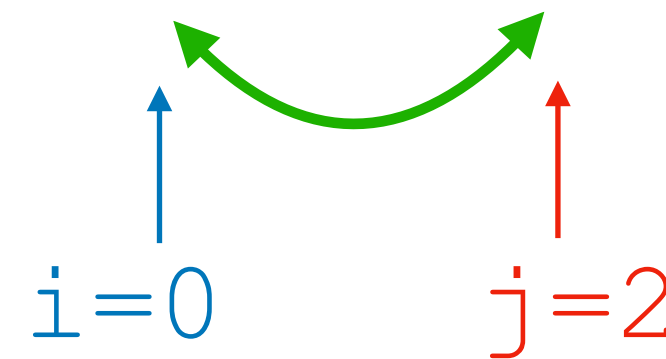
- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```



- Wenn **i==len(liste)-1** gib liste aus

- Wenn **i<len(liste)-1**

- Für alle **Indizes j** $\geq i$:

Schritt 1 1. **vertausche** `liste[i]` **und** `liste[j]`

Schritt 2 2. verfare **rekursiv** mit **i+=1**

Schritt 3 3. mache Vertauschung **rückgängig**

Permutationen einer Liste

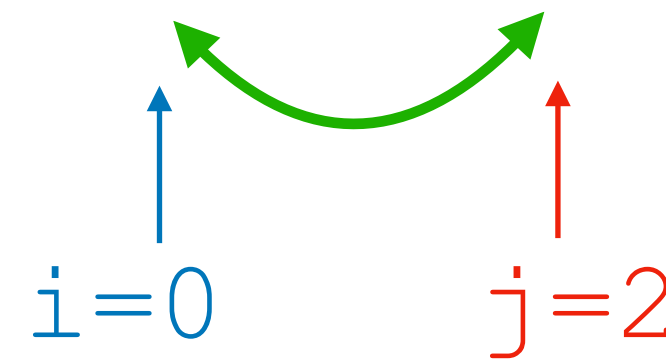
- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[1, 2, 3, 17, 5]
```



- Wenn `i==len(liste)-1` gib liste aus

- Wenn `i<len(liste)-1`

- Für alle **Indizes** `j>=i`:

Schritt 1 1. **vertausche** `liste[i]` **und** `liste[j]`

Schritt 2 2. verfare **rekursiv** mit `i+=1`

Schritt 3 3. mache Vertauschung **rückgängig**

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[1, 2, 3, 17, 5]
```



i=1

- Wenn **i==len(liste)-1** gib liste aus

- Wenn **i<len(liste)-1**

- Für alle **Indizes j** **j>=i**:

Backtracking

Schritt 1

1. **vertausche** liste[i] **und** liste[j]

Schritt 2

2. verfare **rekursiv** mit **i+=1**

Schritt 3

3. mache Vertauschung **rückgängig**

Permutationen einer Liste in Python

```
def permutiere(liste, i):  
    if (i == len(liste)-1):  
        print(liste)  
    else:  
        for j in range(i, len(liste)):  
            liste[i], liste[j] = liste[j], liste[i]  
            permutiere(liste, i+ 1)  
            liste[i], liste[j] = liste[j], liste[i]  
  
permutiere([1, 2, 3, 4, 5, 6], 0)
```

Backtracking

Schritt 1

Schritt 2

Schritt 3

Alle Binärzeichenketten einer gewissen Länge

```
def binaerwoerter(n,wb):  
    if n==0:  
        ausgabe=""  
        keys=list(wb.keys())  
        keys.sort()  
        for i in keys:  
            ausgabe=str(wb[i])+ausgabe  
        print(ausgabe)  
    else:  
        for i in [0,1]:  
            wb[n]=i  
            binaerwoerter(n-1,wb)
```

Schritt 1 auf vorheriger Folie

Schritt 2 auf vorheriger Folie

```
wb={ }  
n=input("Geben Sie eine Wortlänge ein: ")  
binaerwoerter(int(n),wb)
```

Mengen in Python

- Listen erlauben uns eine Sequenz von Elementen zu speichern
- insbesondere dürfen Elemente mehrfach auftauchen
- Listen sind geordnet

- Eine **Menge** ist eine **ungeordnete Sammlung von Objekten**

- Python bietet einen speziellen Mengentyp bereit

```
meineMenge = {1, 2, 3, 5}
```

- Typkonvertierung von einer Liste in eine Menge

```
set([1, 2, 3, 2]) liefert Menge {1, 2, 3}
```

nicht geordnet

- Typkonvertierung von Menge in eine Liste

```
list({1, 2, 5, 4, 3, 17, 1222}) liefert Liste  
[1, 2, 3, 4, 5, 1222, 17]
```

nicht unbedingt geordnet

Modifikation von Mengen

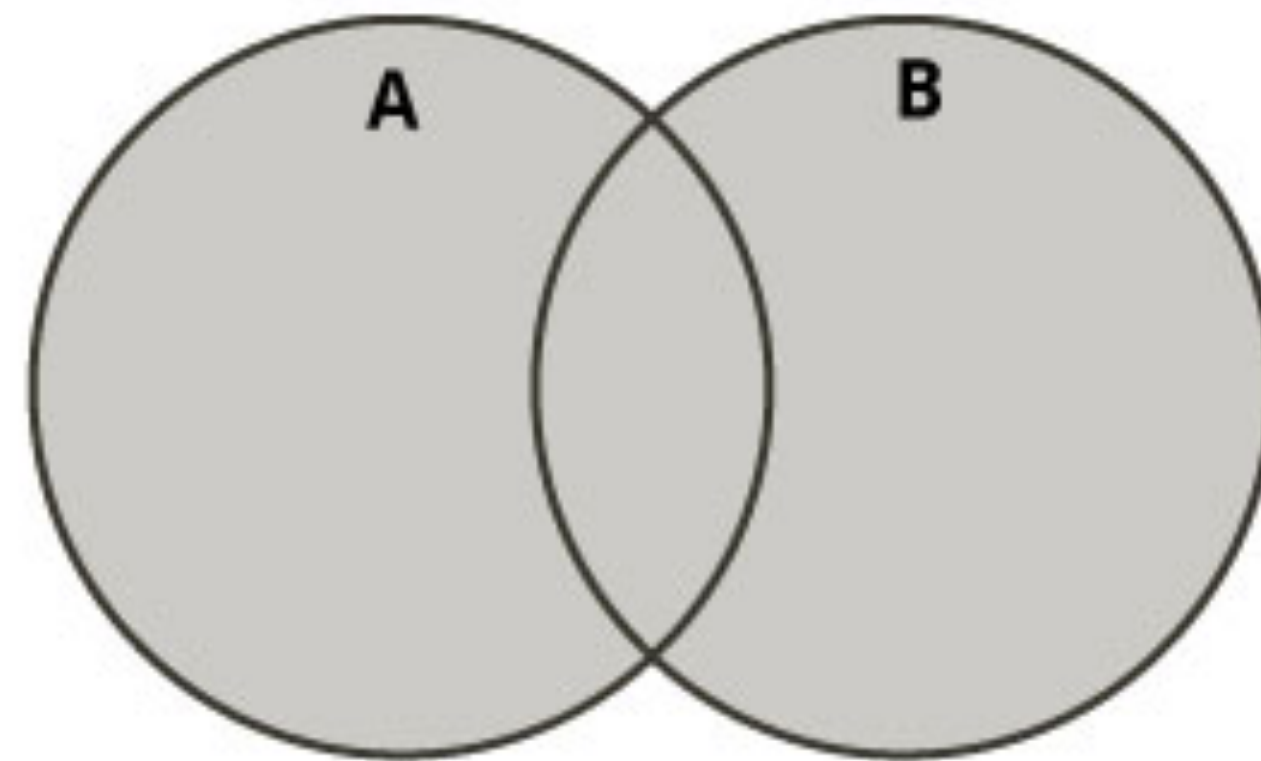
- Die leere Menge kann mittels `meineMenge={ }` initialisiert werden
- Die Elemente in einer Menge müssen nicht vom selben Typ sein
 - `{ 'A' , 1 , 1.2 }` ist also eine gültige Menge in Python
- Ein Element `element` kann mittels `meineMenge.add(element)` hinzugefügt werden
- Es kann sogar eine Liste mittels `meineMenge.update(L)` kann eine Liste `L` hinzugefügt werden
- Mittels `meineMenge.discard(element)` kann ein Element `element` der Menge entfernt werden (keine Fehlermeldung/Exception, wenn Element nicht vorhanden)
- mittels `meineMenge.pop()` wird irgendein Element der Menge `meineMenge` entfernt und zurückgegeben

Operationen auf Mengen

$A = \{1, 2, 3, 4, 5\}$

$B = \{4, 5, 6, 7, 8\}$

- Die **Vereinigung** zweier Mengen A und B ist die Menge aller Elemente die in **mindestens einer der beiden Mengen** vorkommen



- In Python werden mittels `|` zwei Mengen vereinigt

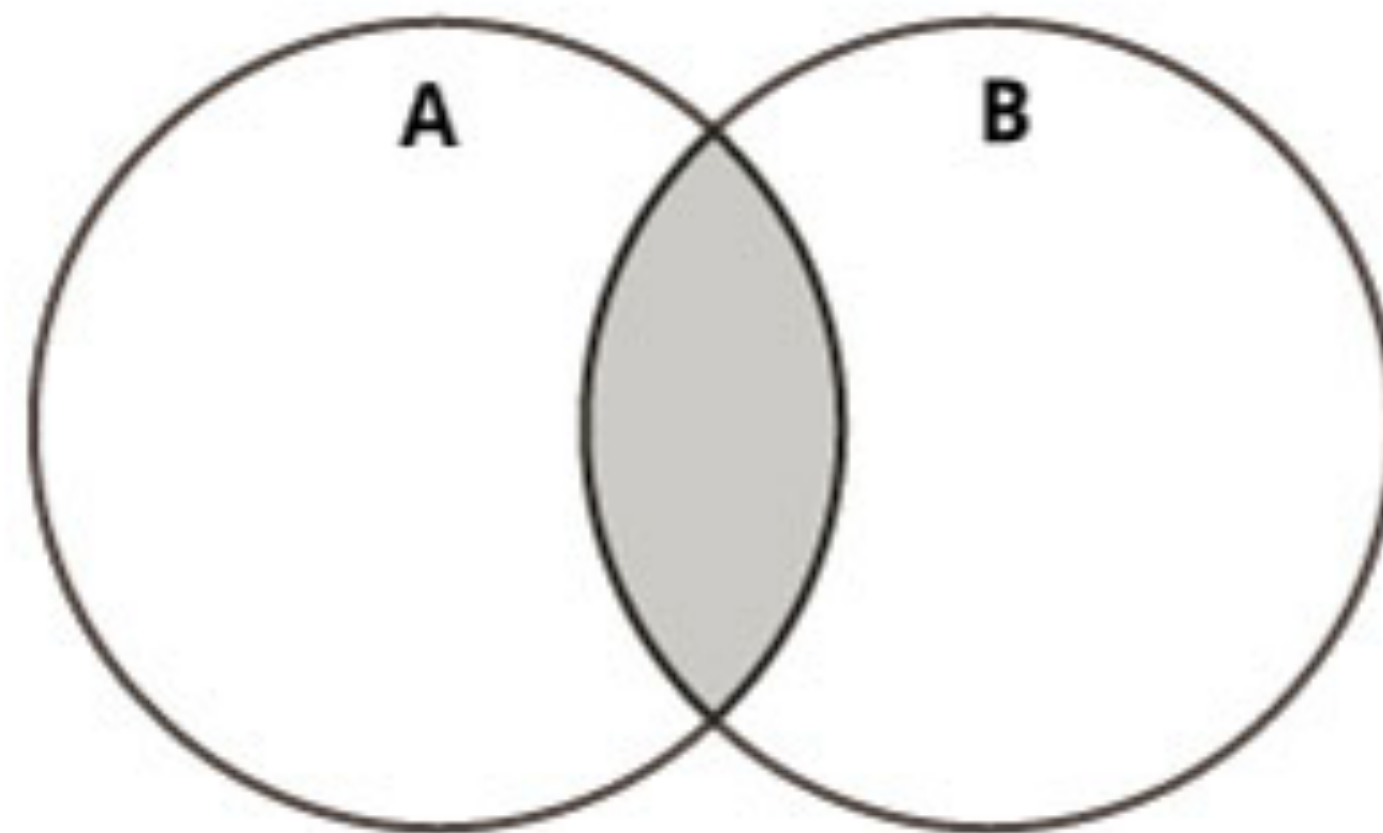
$A \mid B$ liefert also $\{1, 2, 3, 4, 5, 6, 7, 8\}$

Operationen auf Mengen

$A = \{1, 2, 3, 4, 5\}$

$B = \{4, 5, 6, 7, 8\}$

- Die **Durchschnitt** zweier Mengen A und B ist die Menge aller Elemente die in **beiden Mengen** vorkommen



- In Python werden mittels **&** zwei Mengen geschnitten

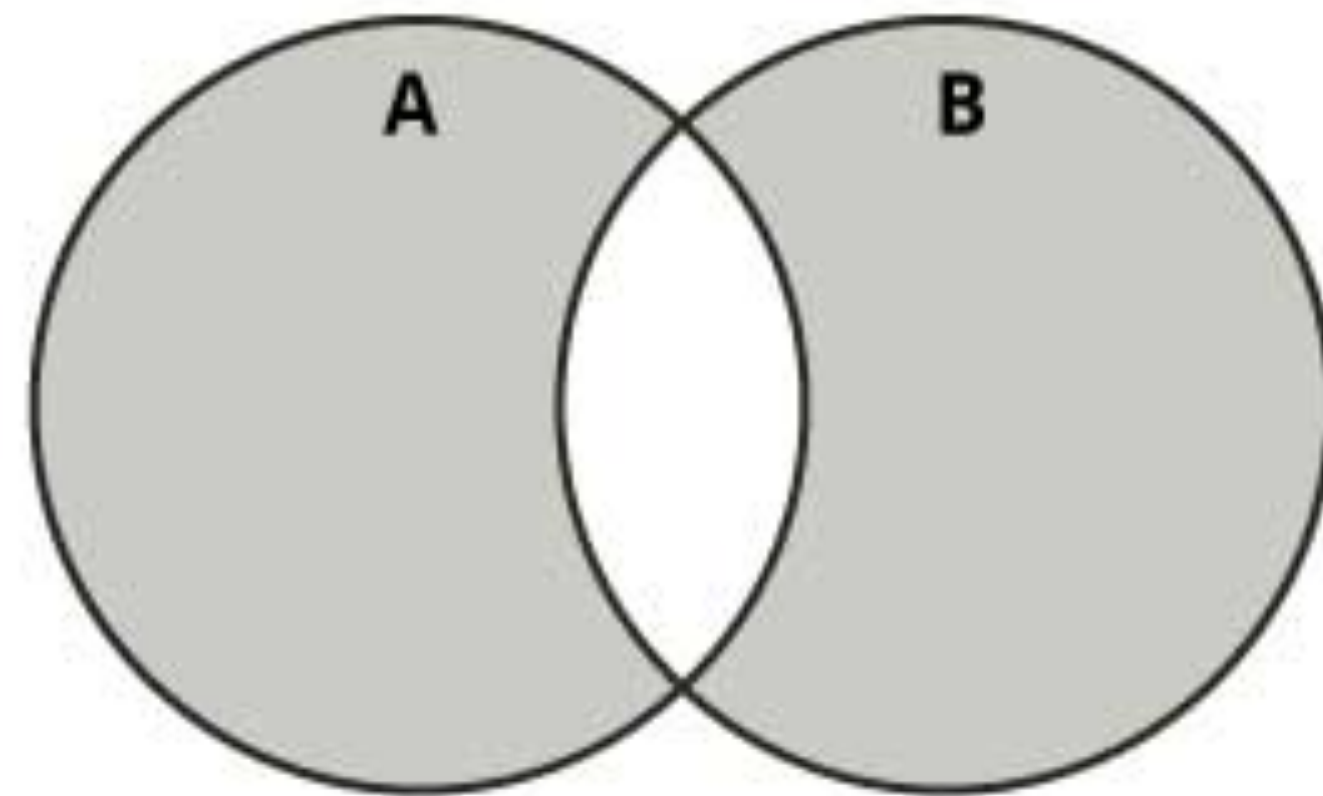
$A \ \& \ B$ liefert also $\{4, 5\}$

Operationen auf Mengen

$$A = \{1, 2, 3, 4, 5\}$$

$$B = \{4, 5, 6, 7, 8\}$$

- Die **symmetrische Differenz** zweier Mengen A und B ist die Menge aller Elemente die **in genau einer der beiden Mengen** vorkommen



- In Python wird dies mittels **$\wedge$** realisiert

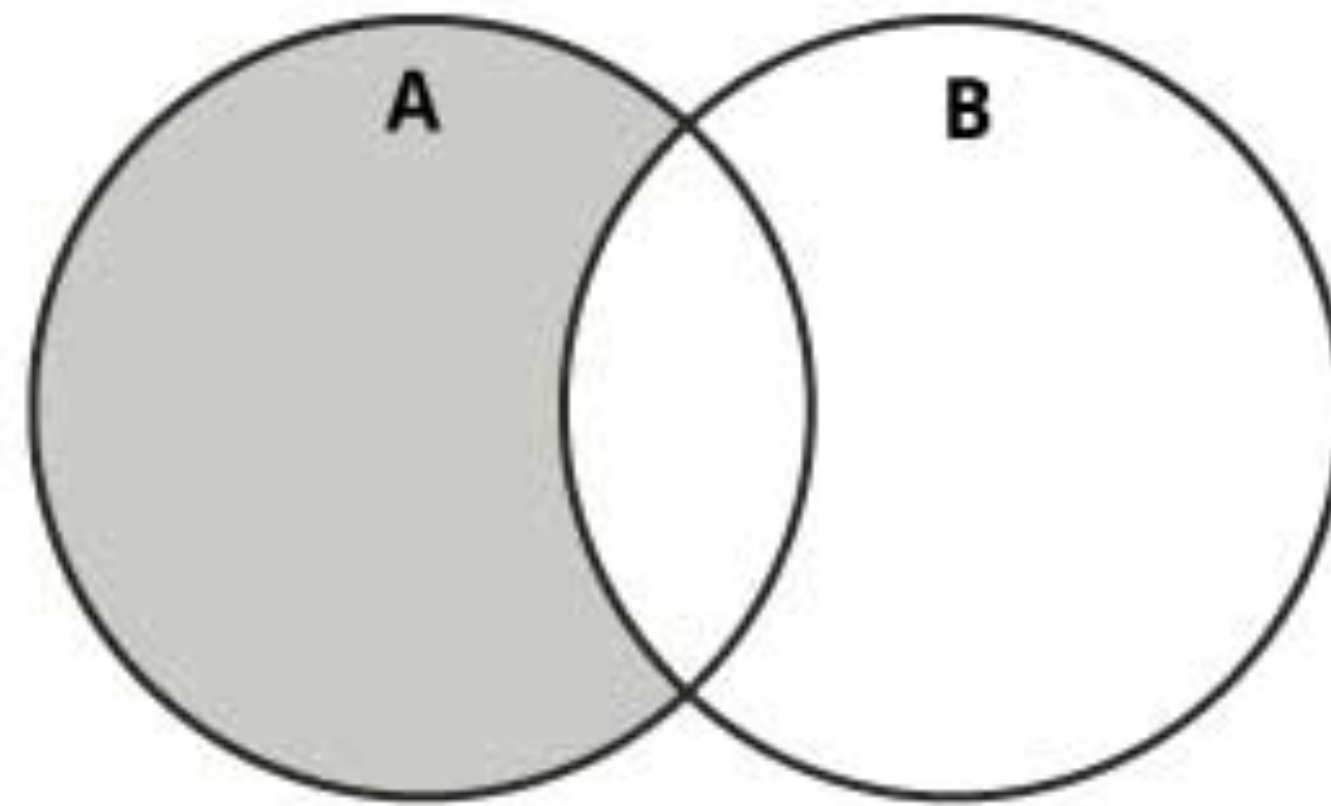
$$A \wedge B \text{ liefert also } \{1, 2, 3, 6, 7, 8\}$$

Operationen auf Mengen

$$A = \{1, 2, 3, 4, 5\}$$

$$B = \{4, 5, 6, 7, 8\}$$

- Die **Differenz** der Menge A von der Menge B ist die Menge aller Elemente die **in A aber nicht in B** vorkommen



- In Python wird dies mittels `-` realisiert

$A - B$ liefert also $\{1, 2, 3\}$

weitere Operationen

- Mitgliedschaft eines Elements `element` in der Menge `meineMenge` kann getestet werden mittels

`element in meineMenge`

vom Typ `bool`

- es kann über eine Menge iteriert werden mittels:
`for e in meineMenge:`

...

Mehr unter:

<https://docs.python.org/2/library/sets.html>

Ein paar nette Abkürzungen: **lambda**

- oftmals will man
 - kurz eine Funktion definieren, die nur einmal verwendet wird
 - Dinge generisch implementieren (d.h. derart implementieren, dass verschiedene Varianten dieser Implementierung verwendet werden)
- Mittel $f = \text{lambda } x, y: x+y$ wird ein Funktionsobjekt der Variablen f zugewiesen - es hat (hier) zwei Argumente x und y und gibt den Wert $x+y$ zurück
- $f(3, 7)$ würde bspw. den Wert 10 liefern

erster Parameter

zweiter Parameter

Rückgabewert

Ein paar nette Abkürzungen:

lambda

```
def sortiere(kleiner,L):  
    trenner = 0  
    while trenner != len(L):  
        for i in range(trenner,len(L)):  
            if kleiner(L[i],L[trenner]):  
                L[trenner], L[i] = L[i], L[trenner]  
        trenner += 1  
    return L
```

```
L = [5,2,1,232,2122,1123,12112121,12,2,434]  
meinKleiner = lambda x,y : x<y  
L=sortiere(meinKleiner,L)
```

Ein paar nette Abkürzungen:

map

- oftmals will man über eine Sequenz (wie z.B. eine Liste) iterieren, Funktion anwenden und das Ergebnis zuweisen
- dies lässt sich mittels

`map (funktion, sequenz)`

realisieren

*liefert ein Objekt
vom Typ `map`*

- Objekte vom Typ `map` lassen sich wiederum in eine Liste oder eine Menge konvertieren

```
def fahrenheit(T):  
    return ((9/5)*T + 32)  
def celsius(T):  
    return (5/9)*(T-32)  
temp = [36.5, 37, 37.5, 39]  
print(temp)  
F = list(map(fahrenheit, temp))  
print(F)  
C = list(map(celsius, F))  
print(C)
```


Ein paar nette Abkürzungen:

map + lambda

- das Programm auf der vorigen Seite hätte man auch knappe schreiben können, indem man **map** und **lambda** kombiniert:

```
Celsius = [39.2, 36.5, 37.3, 37.8]
Fahrenheit = map(lambda x: (9/5)*x + 32, Celsius)
print Fahrenheit
C = map(lambda x: (5/9)*(x-32), Fahrenheit)
print C
```

- es sind auch noch mehr Parameter möglich

```
a = [1, 2, 3, 4]
b = [17, 12, 11, 10]
c = [-1, -4, 5, 9]
print(list(map(lambda x, y: x+y, a, b) ))
print(list(map(lambda x, y, z: x+y+z, a, b, c) ))
print(list(map(lambda x, y, z: x+y-z, a, b, c) ))
```

Ein paar nette Abkürzungen:

filter

- recht knapp kann man auch gewisse Elemente einer Liste entfernen für die die Funktion `true` liefert

filter (funktion, sequenz)

*liefert ein Objekt
vom Typ `filter`*

- Objekte vom Typ `filter` lassen sich wiederum in eine Liste oder eine Menge konvertieren

```
L = [0,1,1,2,3,5,8,13,21,34,55]
ergebnis = list(filter(lambda x: x % 2, L))
print(ergebnis)
ergebnis = list(filter(lambda x: x % 2 == 0, L))
print(ergebnis)
```


List Comprehensions

- liefern eine elegante Möglichkeit Listen zu erzeugen
- sie kombinieren Iteration, Listenerzeugung und Kontrollfluß in einem
- die generelle Form

```
neuesL = [ ausdr(var) for var in L if bed(var) ]
```

liefert eine Liste `neuesL` aller Elemente der Form `ausdr(var)` in der Liste `L`, die die Bedingung `bedingung(var)` erfüllen (`var` ist Laufvariable über Elemente in `L`), ist also zu folgendem Programm äquivalent:

```
neuesL=[]  
for var in L:  
    if bed(var) :  
        neuesL.append(ausdr(var) )
```

List Comprehensions

- Der Ausdruck

```
[i*i*i for i in [0,1,2,3,4,5] if i%2==0]
```

erzeugt die Liste [0, 8, 64]

- Der Ausdruck

```
[x.lower() for x in ["A","B","C"]]
```

erzeugt die Liste ["a", "b", "c"]

- Der Ausdruck

```
[x+y for x in [10,30,50] for y in [20,40,60]]
```

erzeugt die Liste [30, 50, 70, 50, 70, 90, 70, 90, 110]

Dictionary Comprehensions

- liefern eine elegante Möglichkeit Wörterbücher (Dictionaries) zu erzeugen
- sie kombinieren Iteration, Erzeugung von Dictionaries und Kontrollfluß in einem
- Beispiel:

`{(i, j) : set(range(1, 10)) for i in range(9) for j in range(9)}`

geschweifte Klammern!

erzeugt ein Wörterbuch

- mit Schlüsselmenge = alle 2-Tupel von Zahlen von 0 bis 8
- das jedem Schlüssel (i, j) die Menge $\{1, 2, \dots, 9\}$ zuweist (für alle i und j von 0 bis 8)

Sudoku

- ist ein aus lateinischen Quadraten (typischerweise 9x9) bestehendes Logikrätsel folgender Form

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

- **Aufgabe:** Die leeren Kästchen so mit Ziffern (1 bis 9) füllen, so dass:
 - in jedem Kästchen genau eine Ziffer steht
 - in jeder **Zeile** jede Ziffer genau einmal vorkommt
 - in jeder **Spalte** jede Ziffer genau einmal vorkommt
 - in jedem der neun **3x3-Unterquadrate** jede Ziffer genau einmal vorkommt

Darstellung eines Spielstands

- wie stellen wir einen aktuellen „Spielstand“ dar?

```
stand = { (i, j) : set(range(1, 10)) for i in range(9) for j in range(9) }
```

im Folgenden auch
so bezeichnet!

- D.h. Paare (i, j) stellen **Kästchen** dar für alle i und j von 0 bis 8
- Jedem dieser Paare ordnet unser Wörterbuch `stand` eine **Teilmenge von $\{1, 2, \dots, 9\}$** zu (sogenannte **Kästchen-Menge**)
- Zu Beginn ist jede Kästchen-Menge auf **$\{1, 2, \dots, 9\}$ initialisiert**, mit der Idee, dass zu Beginn keine Einschränkung der Kästchen vorhanden ist (alle Zahlen in $\{1, 2, \dots, 9\}$ kommen als Kästcheneintrag in Frage)
- danach initialisieren wir `stand` per Hand, indem wir manche der Mengen einelementig machen (nächste Folie)
- wir meinen mit **Spielstand** im Folgenden immer ein Wörterbuch, das Kästchen auf Teilmengen von $\{1, 2, \dots, 9\}$ abbildet

Initialisierung des Spielstands

- The world's hardest Sudoku:

<https://www.telegraph.co.uk/news/science/science-news/9359579/Worlds-hardest-sudoku-can-you-crack-it.html>

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

```
def init1():  
    b[(0,0)]={8}  
    b[(1,2)]={3}  
    b[(1,3)]={6}  
    b[(2,1)]={7}  
    b[(2,4)]={9}  
    b[(2,6)]={2}  
    b[(3,1)]={5}  
    b[(3,5)]={7}  
    b[(4,4)]={4}  
    b[(4,5)]={5}  
    b[(4,6)]={7}  
    b[(5,3)]={1}  
    b[(5,7)]={3}  
    b[(6,2)]={1}  
    b[(6,7)]={6}  
    b[(6,8)]={8}  
    b[(7,2)]={8}  
    b[(7,3)]={5}  
    b[(7,7)]={1}  
    b[(8,1)]={9}  
    b[(8,6)]={4}
```


Darstellung des Spielstands

wollen wir so ausgeben

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

```
8 * * | * * * | * * *
* * 3 | 6 * * | * * *
* 7 * | * 9 * | 2 * *
-----
* 5 * | * * 7 | * * *
* * * | * 4 5 | 7 * *
* * * | 1 * * | * 3 *
-----
* * 1 | * * * | * 6 8
* * 8 | 5 * * | * 1 *
* 9 * | * * * | 4 * *
```

Ausgabe des Spielstands

```
def ausgabe(stand):  
    for i in range(9):  
        tmp=""
```

```
        if i==3 or i==6:
```

```
            print("-"*30)
```

```
        for j in range(9):
```

```
            if j==3 or j==6:
```

```
                tmp+=" | "
```

```
            d=stand[(i,j)]
```

```
            if len(d)==1:
```

```
                for e in d:
```

```
                    tmp+=" "+str(e)
```

```
            else:
```

```
                tmp+=" "+str(d)+" "
```

```
        print(tmp)
```

...ansonsten geben wir * aus

Grenze des 3x3 Unterquadrats bei Zeile
3 und 6 (der Zeilen 0 bis 8)

horizontale Linie, um Grenze des
3x3 Unterquadrats anzuzeigen

Grenze des 3x3 Unterquadrats
bei Spalte 3 und 6 (der Spalten 0 bis 8)

vertikale Linie, um Grenze des
3x3 Unterquadrats anzuzeigen

lese Menge des aktuellen
Kästchens aus

Kästcheninhalt wird nur ausgegeben
wenn Menge in Kästchen einelementig ist...

Ziel des Spiels als Mengen

- Nach der Initialisierung (wie mit dem oben abgebildeten “world’s hardest Sudoku”) sind die Mengen der Kästchen also entweder
 - nach wie vor **{1,2,...,9}** (wenn das Kästchen der Initialposition leer war)
 - einelementig, also von der Form **{k}** für ein k in $\{1,2,...,9\}$ (wenn das Kästchen gefüllt war)

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

hier bspw. {4}

- **grobe Idee des Sudoku-Lösers:** Entferne systematisch Kästchenmengen mit mindestens zwei Elementen so, dass am Ende nur noch einelementige Kästchenmengen vorhanden sind und die Sudoku-Bedingungen nicht verletzt werden

Kopie eines Spielstands

- da wir Sudoku mittels Backtracking lösen wollen, wird es notwendig sein vom aktuellen Spielstand eine Kopie anzufertigen

```
def kopie(stand):  
    return { (i,j): stand[(i,j)].copy() \  
            for i in range(9) for j in range(9) }
```

Zeilenumbruch

kopiert die Liste in `stand[(i,j)]`

„Bürsten“ von Spielständen

- Durch Anwendung der Sudoku-Bedingungen können wir u.U. einige der Kästchen-Mengen kleiner machen:

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

statt $\{1,2,3,4,5,6,7,8,9\}$ sollte diese Menge
weder eine 3 noch eine 6 noch eine 8
noch eine 2 enthalten!

- Wenn wir gewisse Mengen kleiner machen, können aus diesen Verkleinerungen weitere Verkleinerungen folgen!
- Wir sollten so lange verkleinern bis es nicht mehr möglich ist („bürsten“) und erst dann systematisch alle Möglichkeiten durchprobieren!

Beeinflussung von Kästchen

- wie können wir Abhängigkeit ausdrücken?

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

Kästchen (1,8) kann u.U. von folgenden Kästchen abhängig sein:

- in der gleichen Zeile:
 $\{(1,j) \mid j \in \{0,1,2,3,4,5,6,7\}\}$

- in der gleichen Spalte:
 $\{(i,8) \mid i \in \{0,2,3,4,5,6,7,8\}\}$

- im gleichen Unterquadrat:
 $\{(0,6),(0,7),(0,8),(1,6),(1,7),(2,6),(2,7),(2,8)\}$

- wie drückt man aus, ob eine Position (x, y) von Position (i, j) abhängig sein kann?

```
def abhaengig(i, j, x, y):  
    return (not((i==x) and (j==y))) and  
           ((i==x) or (j==y) or (i//3==x//3) and (j//3==y//3))
```


Umsetzung von Bürsten

```
def buersten(sudoku):  
    def abhaengig(i,j,x,y):  
        return (not((i==x) and (j==y))) and  
                ((i==x) or (j==y) or (i//3==x//3) and (j//3==y//3))  
    sudokuk=kopie(sudoku)  
    entfernt=False
```

Anlegen einer Kopie

noch kein Element einer Menge
eines Kästchens entfernt

```
effects=[(wert,x,y) for i in range(9)\  
                for j in range(9)\  
                for x in range(9)\  
                for y in range(9)\  
                for wert in sudokuk[(i,j)]\  
                if len(sudokuk[(i,j)])==1 and\  
                wert in sudokuk[(x,y)] and\  
                abhaengig(i,j,x,y)]
```

Zeilenumbruch

```
for (wert,x,y) in effects:  
    if wert in sudokuk[(x,y)]:  
        sudokuk[(x,y)].remove(wert)  
        entfernt=True  
if entfernt:  
    sudokuk=buersten(sudokuk)  
return sudokuk
```

Sammele Tripel (wert,x,y) ein,
wobei wert in Kästchen (x,y)
vorkommt und nachweislich gelöscht
werden muss.

Lösen durch Backtracking

1. **initialisiere** das zu lösende Sudoku durch unsere Darstellung von Spielständen (d.h. Kästchen-Mengen sind entweder **einelementige Kästchenmengen** oder **$\{1,2,\dots,9\}$**) in Spielstand `stand`
2. Falls eine Kästchenmenge in `stand` leer ist, gib **False** zurück
3. Falls alle Kästchen in `stand` einelementig sind, gib `stand` aus und gib dann **True** zurück
4. Lege eine **Liste** `nonsingletons` von Tripeln $(i, j, \text{stand}[(i, j)])$ an, wobei
 - (i, j) ein Kästchen ist
 - Kästchenmenge `stand[len(i, j)]` mindestens zwei Elemente hat
5. **Sortiere** `nonsingletons` **absteigend** nach Größe der Kästchenmengen
6. Entferne letztes Element $(i, j, \text{ziffern})$ von `nonsingletons` und kopiere `ziffern` in `ziffernkopie` (damit keine Nebeneffekte auftreten)
7. Für alle Ziffern `k` in `ziffernkopie`:
 - A. Lege Kopie von `stand` an (damit keine Nebeneffekte auftreten)
 - B. Setze `stand[(i, j)] = {k}`
 - C. Löse Problem rekursiv mit diesem neuen `stand` (d.h. gehe zu 2.)
 - D. Falls Rückgabewert
 - **True**, gib **True** zurück
 - **False**, setze `stand` auf Kopie zurück und versuche nächste Ziffer (d.h. gehe zu 7.)
8. gib **False** zurück

Backtracking-Schritt

Lösen durch Backtracking

```
def erfuellbar(stand):
    if len([stand[(i,j)] for i in range(9) for j in range(9) \
            if len(stand[(i,j)])==0])>=1: return False
    elif len([stand[(i,j)] for i in range(9) for j in range(9) \
            if len(stand[(i,j)])==1])==81:
        print("Wir haben eine Loesung gefunden")
        ausgabe(stand)
        return True
    else:
        nonsingletons=[(i,j,stand[(i,j)]) \
                        for i in range(9) \
                        for j in range(9) \
                        if len(stand[(i,j)])>1]
        cmp = lambda a,b: len(a[2])>len(b[2])
        nonsingletons = sortsuite.sortiere(cmp,nonsingletons)
        (i,j,ziffern)=nonsingletons.pop()
        ziffernkopie=ziffern.copy()
        old=stand.copy()
        for x in ziffernkopie:
            stand[(i,j)].clear()
            stand[(i,j)].add(x)
            stand=buersten(stand)
            if erfuellbar(stand):
                return True
            stand=old.copy()
        return False
```


Das Hauptprogramm

```
import sortsuite
```

```
def ausgabe...
```

```
def kopie...
```

```
def buersten...
```

```
def erfuellbar...
```

```
def init1...
```

```
stand={ (i,j):set(range(1,10)) for i in range(9) for j in range(9) }
```

```
init1()
```

```
print("Wir wollen folgendes Sudoku loesen")
```

```
ausgabe(stand)
```


Rekursion in der Informatik...

- ... ist eine Art und Weise Objekte/Probleme zu beschreiben
(zum Beispiel wohlgeklammerte Ausdrücke)
 - ... ist eine Art und Weise Lösungen zu entwerfen
(zum Beispiel für Fakultätsberechnung und Türme von Hanoi)
-
- ... ist genauso mächtig wie Iteration
 - ... erlaubt oftmals elegante Lösungen für Probleme
 - ... ist von zentraler Bedeutung in der Berechenbarkeitstheorie
(was kann von Computern berechnet werden)
 - ...wird von einigen Programmiersprachen verlangt wie
z.B. in Scheme, LISP, ML, Haskell (keine Schleifen möglich)

Die O-Notation



- 1894 eingeführt vom deutschen Zahlentheoretiker Paul Bachmann
- Die O-Notation (großes O) gibt eine obere Schranke für das asymptotische Wachstum einer Funktion
- **Gross O oder $O()$** wird im Rahmen einer Worst-Case-Analyse angewendet
 - Wenn der schlechteste Fall eintritt, ist dieser der Flaschenhals einer Programmberechnung
 - drückt das Wachstum relativ zur Eingabelänge aus
 - bewertet **weder** die Maschine **noch** die Implementierung

Die O-Notation, formal

Seien $f, g : \mathbb{N} \rightarrow \mathbb{N}$ jeweils Funktionen von $\mathbb{N}$ nach $\mathbb{N}$.

Wir schreiben $f \in O(g)$ falls Konstanten $c \in \mathbb{R}^+$ und $n_0 \in \mathbb{N}$ existieren so, dass für alle $n \in \mathbb{N}$ mit $n > n_0$ gilt:

$$f(n) \leq c \cdot g(n)$$

Wir sagen dann „ f ist/liegt in $O(g)$ “

Beispiele:

- Seien $f(n) = 2n$ und $g(n) = n$ für alle $n \in \mathbb{N}$.

Dann gilt $f \in O(g)$ denn wir wählen $c = 2$ und $n_0 = 0$.

- Seien $f(n) = 4n + 4$ und $g(n) = n^2$ für alle $n \in \mathbb{N}$.

Dann gilt $f \in O(g)$ denn wir wählen $c = 1$ und $n_0 = 5$

und es gilt $4n + 4 \leq 1 \cdot n^2$ für alle $n > 5 = n_0$.

Wir schreiben der Einfachheit halber
gleich $O(n)$

Genaueres Wachstum vs $O()$

```
def fakultaet_iterativ(n):  
    antwort = 1  
    while n >= 1:  
        antwort *= n  
        n -= 1  
    return antwort
```

*antwort = antwort * n*
temp = n-1
n=temp

- Berechnet die Fakultätsfunktion
- Anzahl der Operationen: $1+5n+1$
- Asymptotische Worst-Case Komplexität: $O(n)$
 - additive Konstanten werden vernachlässigt
 - multiplikative Konstanten werden vernachlässigt

Was ist Programmeffizienz?

- Computer werden immer schneller: Vielleicht ist Effizienzanalyse daher irgendwann unwichtig?
 - Datenmengen können jedoch sehr gross werden (Google hat im Jahre 2014 30 Billionen Webseiten verwaltet, was rund 100 Millionen Gigabyte entspricht)
 - Einfache algorithmische Lösungen **skalieren nicht** mit noch größeren Datenmengen
 - Wichtige Frage daher: Nach welchen Kriterien ist ein Programm effizient?
 - Wir trennen **Zeit-** und **Speichereffizienz** eines Programms
 - Kompromiss manchmal nötig:
 - Verwende Speicher um bereits Berechnetes zu sichern (siehe Fibonacci mit einem Wörterbuch)
 - wir fokussieren uns auf Zeiteffizienz
- Gewisse Zusammenhänge bestehen doch:
- Speicherbedarf $\leq$ Zeitbedarf
 - Zeitbedarf ist maximal exponentiell im Speicherbedarf

Was ist Programmeffizienz?

Es gibt gewisse Herausforderungen Effizienz eines algorithmischen Problems zu verstehen:

- Für einen Algorithmus gibt es **verschiedene Implementierungen**
- Für jedes Problem gibt es **verschiedene Algorithmen**, die man einsetzen könnte
- Wir wollen Implementierungen und abstrakte Algorithmen stets trennen

Messung von Programmeffizienz

Vorschläge:

- durch eine Stoppuhr
- die Anzahl der Operationen zählen
- abstraktes Größenwachstum

wir werden uns noch überzeugen, dass dies die angemessene Art und Weise ist

- zu messen welche Auswirkung die Wahl eines Algorithmus für ein gegebenes Problem hat und
- zu messen welche inhärente algorithmischen Schwierigkeiten einem gegebenen Problem zugrundeliegen.

Ausführungszeiten stoppen

- wir binden das `time`-Modul ein
- wir können die Methoden und Funktionen des Moduls `time` ins eigene Programm einbinden

- Starte die Stoppuhr

```
import time
```

- Rufe Funktion auf

```
def f(c):  
    return c*9/5+32
```

- Halte Stoppuhr an

```
t0 = time.process_time()  
  
f(10000)  
t1 = time.process_time() - t0  
print("Benötigte Zeit ",t1,"Sekunden")
```


Stoppuhrmessung ist inkonsistent

- Ziel: Bewertung verschiedener Algorithmen
- Laufzeit verschiedener Algorithmen **variiert** ✓
- Laufzeit verschiedener Implementierungen **variiert** ✗
- Laufzeit verschiedener Computer **variiert** ✗
- Laufzeit ist **nicht einmal vorhersagbar** bei kleinen Eingaben ✗
- Laufzeit **variiert** bei unterschiedlichen Eingaben **ohne** dass dadurch ein **Bezug** zwischen Eingabegröße und Laufzeit ableitbar ist ✗
- Schlimmer noch: Laufzeit bei fester Eingabe **nicht reproduzierbar** ✗

Zählen von Operationen

- wir nehmen der Einfachheit halber an, dass folgende Operationen konstante Zeit benötigen
 - mathematische Operationen
 - Vergleiche
 - Zuweisungen
 - Zugriff auf Objekte im Hauptspeicher
- Zähle die Anzahl der ausgeführten Operationen als Funktion der Eingabelänge

```
def f(c):  
    return c*9/5+32
```

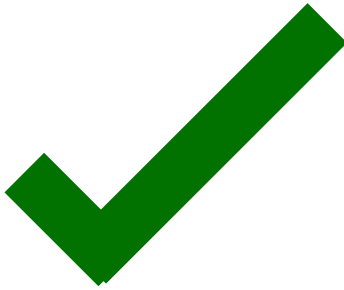
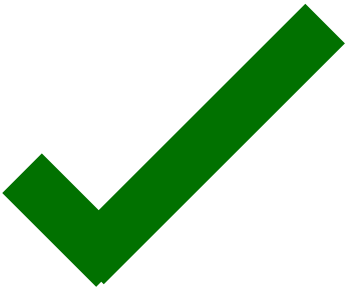
3 Operationen

```
def meinesumme(x):  
    total = 0  
    for i in range(x+1):  
        total += i  
    return total
```

1 Operation
x Durchläufe
1 Operation
2 Operationen

meinesumme hat $3x+1$ Operationen

Operationen zählen ist gut aber...

- Ziel: Bewertung verschiedener Algorithmen 
- Laufzeit verschiedener Algorithmen **variiert**
- Laufzeit verschiedener Implementierungen **variiert** 
- **unklar** welche Operationen gezählt werden sollen 
- Laufzeit **variiert** bei unterschiedlichen Eingaben **so, dass** dadurch jedoch ein **Bezug** zwischen Eingabegröße und Laufzeit ableitbar ist 
- Laufzeit bei fester Eingabe **reproduzierbar** 

Wir benötigen eine noch bessere Lösung

- Stoppuhrmessung und Zählen **bewertet Implementierungen**
- Stoppuhrmessung bewertet **Maschinen**
- Wir wollen **Algorithmen bewerten**
- Wir wollen **Skalierbarkeit bewerten**
- Wir wollen anhand der **Eingabelänge bewerten**

Wir benötigen eine noch bessere Lösung

- Wir fokussieren uns immer noch darauf Operationen zu zählen aber sorgen uns **nicht um Implementierungsdetails** (z.B. ob wir 3 oder 4 elementare Operationen innerhalb einer Schleife durchlaufen)
- Wir analysieren was passiert, wenn Größe des Problems **beliebig groß** wird
- Wir wollen die **Laufzeit** bis zum Ende der Berechnung messen und diese **in Abhängigkeit zur Eingabe- bzw. Problemgröße** darstellen.
- Wir müssen uns entscheiden was wir genau messen wollen, da uns (manchmal) gewisse Details nicht interessieren

Was wollen wir messen?

- Wir wollen die Effizienz in Abhängigkeit der Eingabelänge messen, also müssen wir festlegen was die Eingabelänge genau ist

- Kann eine natürliche Zahl sein

`meinesumme(x)`

- Kann die Länge einer Liste sein

`listensumme(liste)`

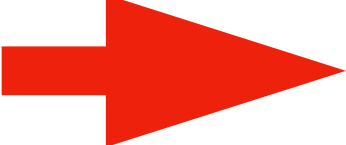
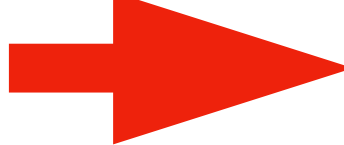
- Gewisse Entscheidungsoptionen möglich, bspw. bei mehreren Eingaben

`suche_nach_element(liste, element)`

Verhalten des Programms hängt von der Eingabe ab

- Sei folgende Funktion gegeben, die nach einem Element in einer Liste sucht

```
def suche_nach_element(liste, element)
    for i in liste:
        if i==element:
            return True
    return False
```

- Wenn `element` das erste Element von `liste` ist  **BEST CASE**
- Wenn `element` nicht in `liste` ist  **WORST CASE**
- Wenn man ungefähr die Hälfte aller Elemente von `liste` durchlaufen muss, um `element` zu finden  **AVERAGE CASE**
- Wir wollen eine allgemeine Laufzeitanalyse durchführen

Best, Average und Worst Case

- Nehmen wir eine Liste L der Länge $\text{len}(L)$ an
- Best Case: Die minimale Laufzeit über alle möglichen Eingaben einer gewissen Länge, nämlich $\text{len}(L)$:
 - Konstante Zeit für Funktion `suche_nach_element`
 - wird immer durch das erste Element der Liste erreicht
- Average Case: Durchschnittslaufzeit gemittelt über alle Eingaben einer gewissen Länge, nämlich $\text{len}(L)$ Unser Hauptaugenmerk wird hierauf gerichtet
 - kann recht aussagekräftiges und realistisches Maß sein
- Worst Case: Maximallaufzeit über alle Eingaben einer gewissen Länge, nämlich $\text{len}(L)$
 - linear in Listenlänge für Funktion `suche_nach_element`
 - die komplette Liste muss notfalls durchlaufen werden

Ordnungen des Größwachstums

- die Programmeffizienz soll bewertet werden, wenn die Eingabelänge **sehr groß** bzw. **beliebig groß** ist
- das Wachstum der Programmlaufzeit soll **in Abhängigkeit der Eingabelänge** gemessen werden
- es soll eine **obere Schranke** angegeben werden, die so klein wie möglich ist
- wir müssen nicht sonderlich genau sein, die **Größenordnung** reicht völlig aus, genaue Werte sind unwichtig
- wir sollen herausfinden können **welcher Teil** des Programms zur Maximallaufzeit beiträgt
- **In einem Satz: Wir wollen eine möglichst klein wachsende Größenordnung für das Worst-Case Verhalten eines Programms in Abhängigkeit der Eingabelänge ermitteln**

Die O-Notation



- 1894 eingeführt vom deutschen Zahlentheoretiker Paul Bachmann
- Die O-Notation (großes O) gibt eine obere Schranke für das asymptotische Wachstum einer Funktion
- **Gross O oder $O()$** wird im Rahmen einer Worst-Case-Analyse angewendet
 - Wenn der schlechteste Fall eintritt, ist dieser der Flaschenhals einer Programmberechnung
 - drückt das Wachstum relativ zur Eingabelänge aus
 - bewertet **weder** die Maschine **noch** die Implementierung

Die O-Notation, formal

Seien $f, g : \mathbb{N} \rightarrow \mathbb{N}$ jeweils Funktionen von $\mathbb{N}$ nach $\mathbb{N}$.

Wir schreiben $f \in O(g)$ falls Konstanten $c \in \mathbb{R}^+$ und $n_0 \in \mathbb{N}$ existieren so, dass für alle $n \in \mathbb{N}$ mit $n > n_0$ gilt:

$$f(n) \leq c \cdot g(n)$$

Wir sagen dann „ f ist/liegt in $O(g)$ “

Beispiele:

- Seien $f(n) = 2n$ und $g(n) = n$ für alle $n \in \mathbb{N}$.

Dann gilt $f \in O(g)$ denn wir wählen $c = 2$ und $n_0 = 0$.

- Seien $f(n) = 4n + 4$ und $g(n) = n^2$ für alle $n \in \mathbb{N}$.

Dann gilt $f \in O(g)$ denn wir wählen $c = 1$ und $n_0 = 5$

und es gilt $4n + 4 \leq 1 \cdot n^2$ für alle $n > n_0 = 5$.

Wir schreiben der Einfachheit halber
gleich $O(n)$

Genaueres Wachstum vs $O()$

```
def fakultaet_iterativ(n):  
    antwort = 1  
    while n >= 1:  
        antwort *= n  
        n -= 1  
    return antwort
```

- Berechnet die Fakultätsfunktion
- Anzahl der Operationen: $5n+2$
- Asymptotische Worst-Case Komplexität: $O(n)$
 - additive Konstanten werden vernachlässigt
 - multiplikative Konstanten werden vernachlässigt

Was genau misst $O(n)$?

- uns interessiert das asymptotische Wachstum der Laufzeit während die Eingabegröße anwächst
- falls wir also einen Ausdruck (wie $5n+2$) für die Anzahl der benötigten Operationen vor uns haben, wollen wir wissen wie dieser Ausdruck asymptotisch wächst, wenn die Eingabegröße (n) anwächst
- daher fokussieren wir uns auf jenen Teil des Terms, der asymptotisch am stärksten anwächst
- wir ignorieren multiplikative und additive Konstanten, da diese unabhängig von der Eingabegröße sind

Beispiele für Vereinfachungen

- entferne Konstanten und multiplikative Faktoren
- der Fokus liegt auf den asymptotisch am stärksten anwachsenden Termen

$$n^2 + 2n + 3$$

$$\in O(n^2)$$

$$n^2 + 1002n + 3^{700000}$$

$$\in O(n^2)$$

$$\log(n) + n + 4$$

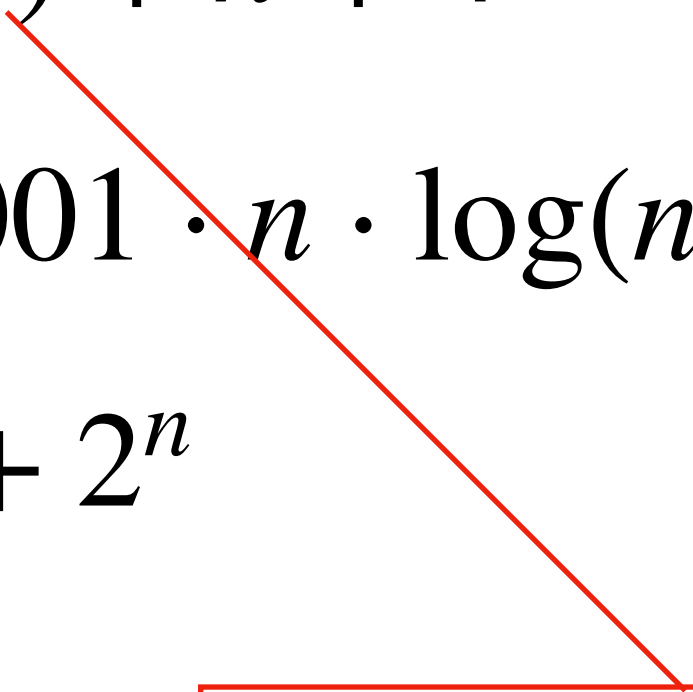
$$\in O(n)$$

$$0,00001 \cdot n \cdot \log(n) + 300n$$

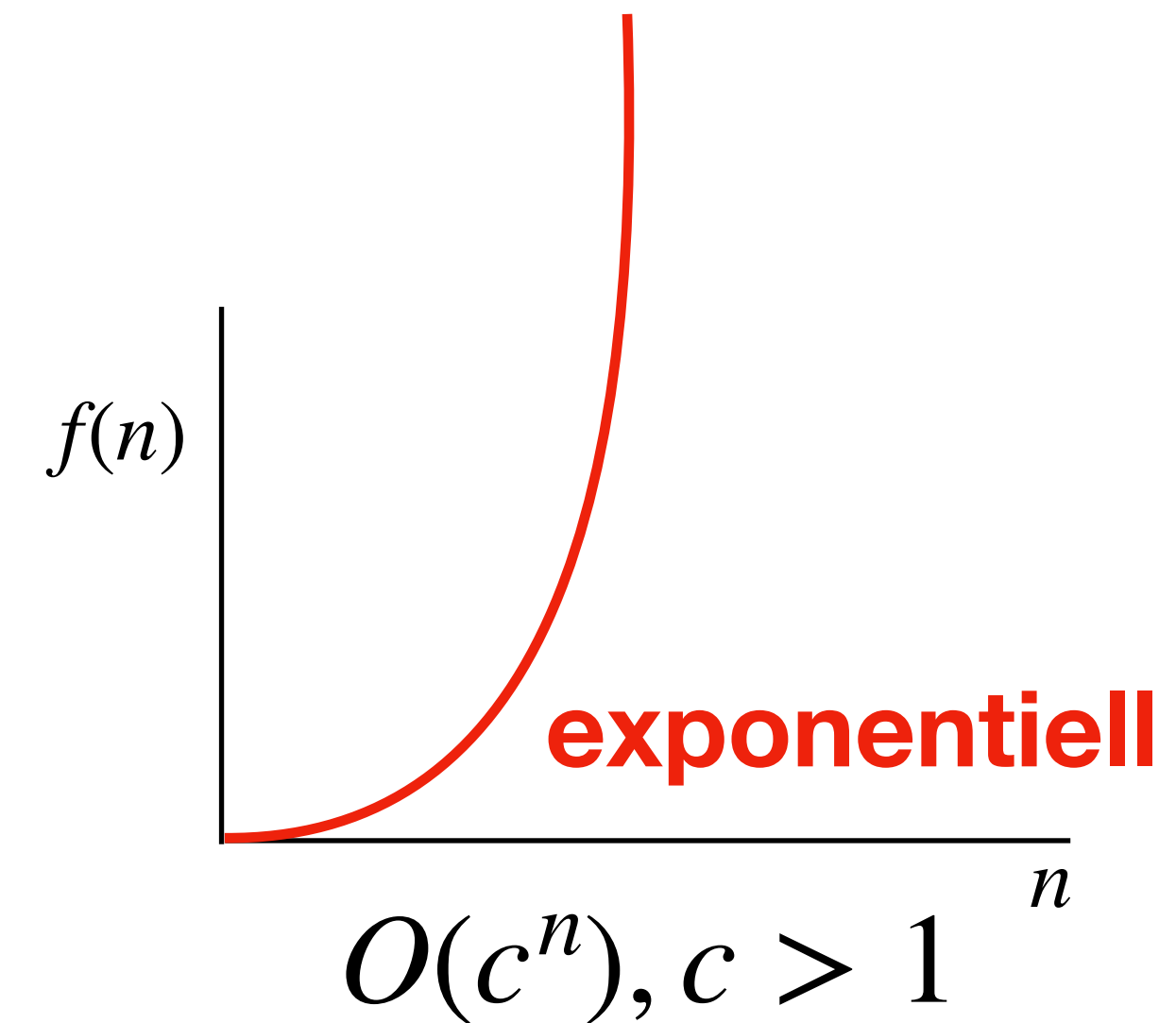
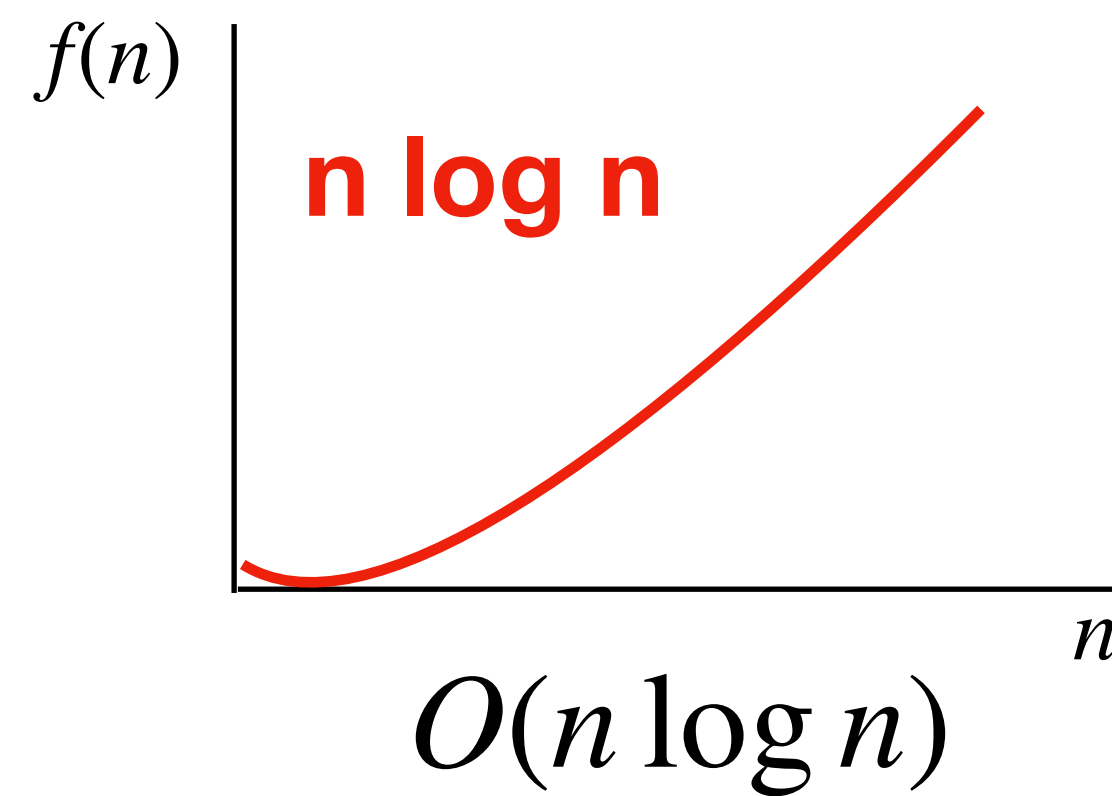
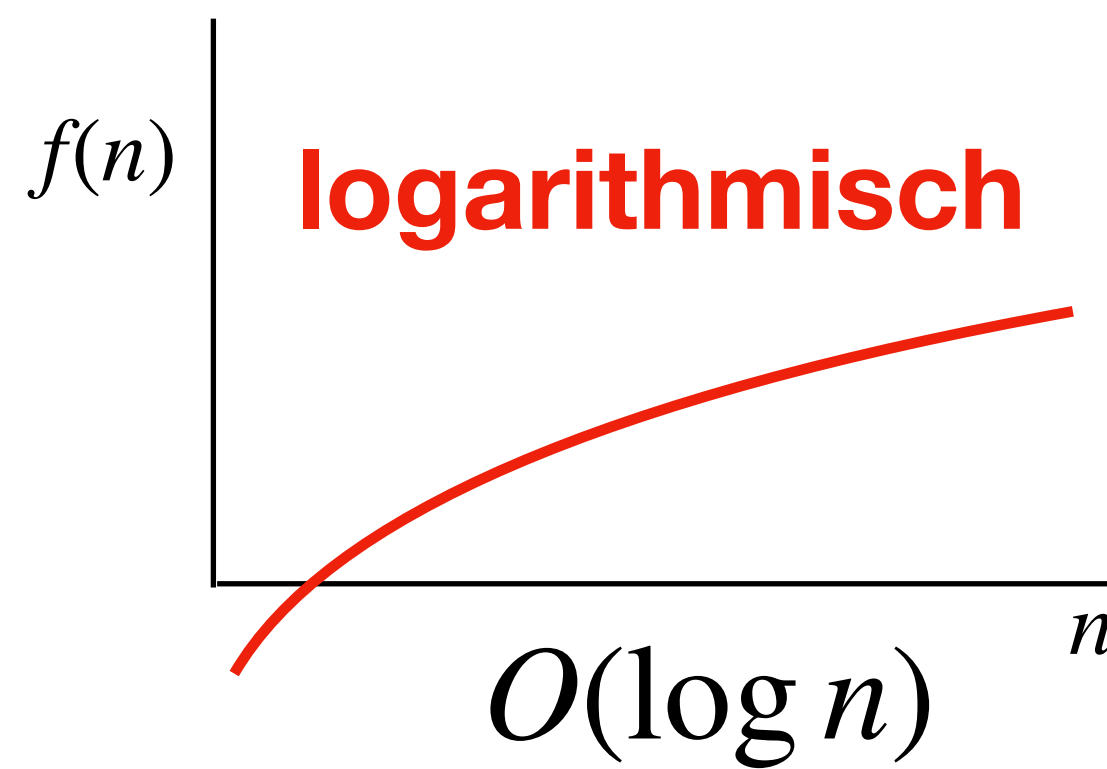
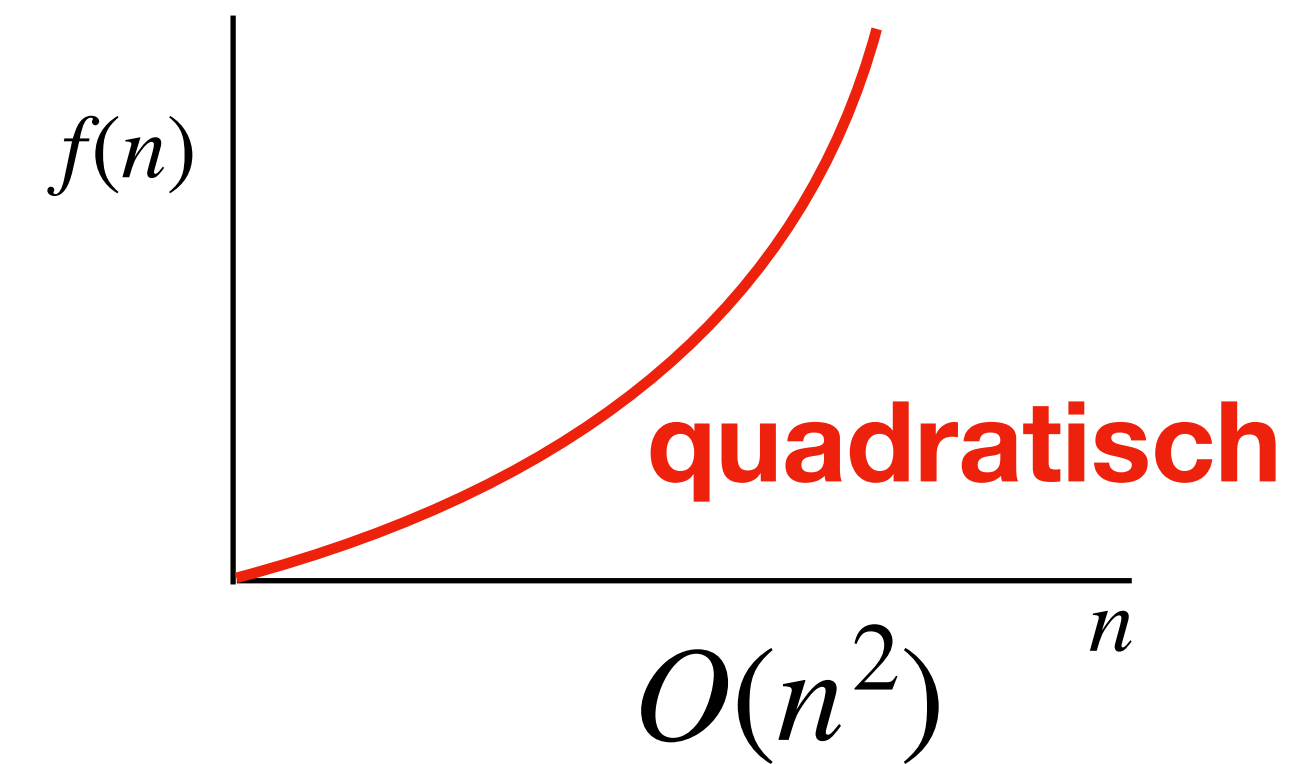
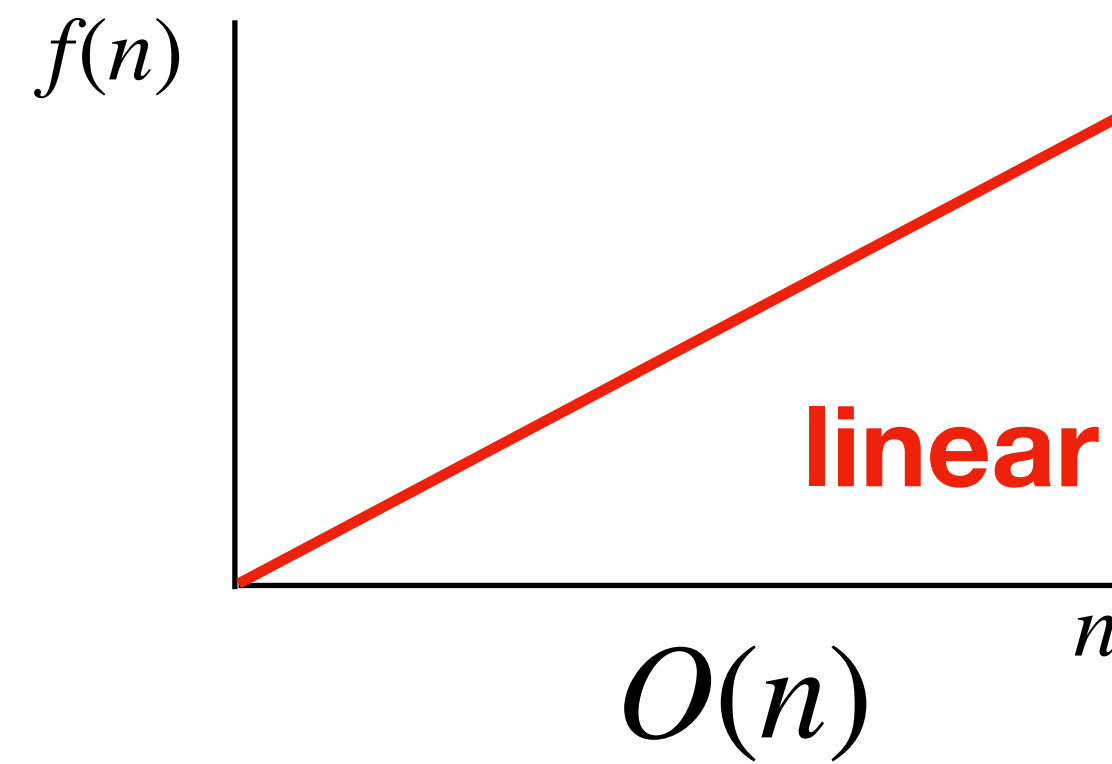
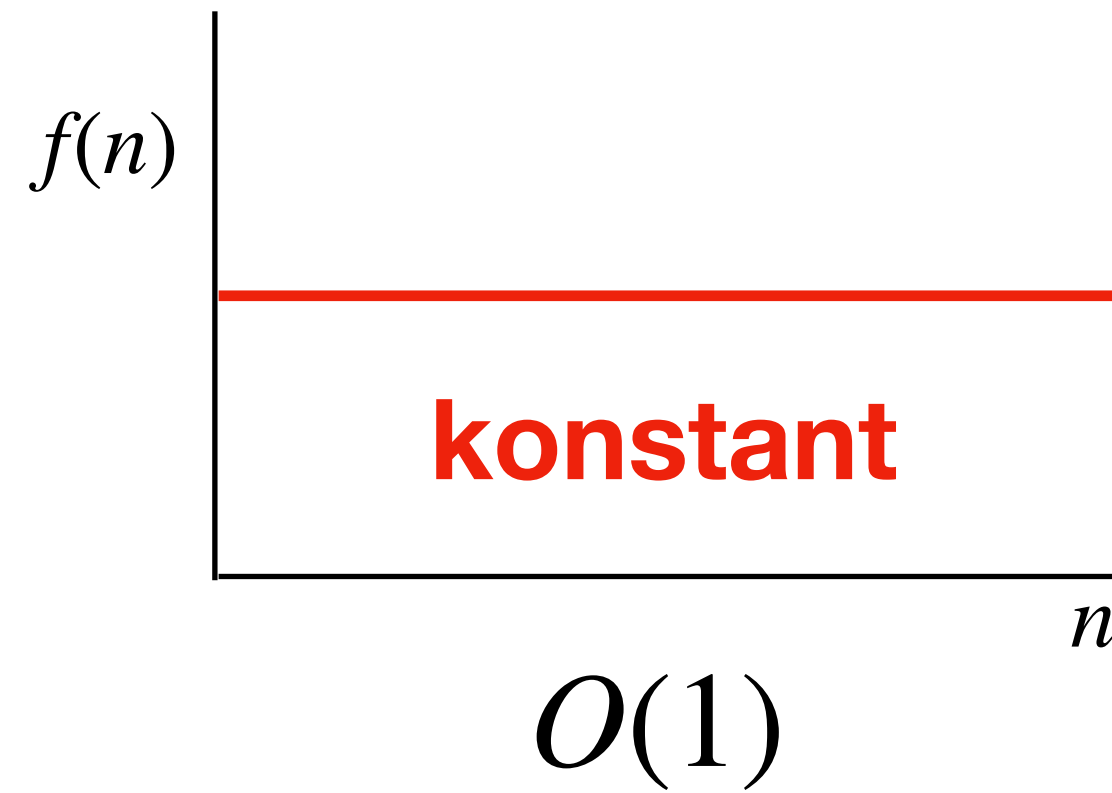
$$\in O(n \log n)$$

$$n^{300} + 2^n$$

$$\in O(2^n)$$


$$\log(n) = \min\{m \mid 2^m \geq n\}$$

Arten von Größenwachstum



Programme und ihre Komplexität analysieren

- **kombiniere** asymptotische Verhaltensweisen
 - Analysiere Verhaltensweisen innerhalb von Funktionen
 - wende gewisse Regeln an, Fokus auf dominanten Termen

Additionsgesetz für $O()$:

- wird oftmals für sequentiell Verhaltensweisen angewendet
- $O(f(n)) + O(g(n)) = O(f(n) + g(n))$
- Zum Beispiel:

```
for i in range(n):  
    print("a")  
for j in range(n*n):  
    print("b")
```

$O(n)$
 $O(n^2)$ } $O(n^2) + O(n) = O(n^2)$

ist $O(n^2) + O(n) = O(n^2)$ wegen des dominanten Terms

Programme und ihre Komplexität analysieren

- **kombiniere** asymptotische Verhaltensweisen
 - Analysiere Verhaltensweisen innerhalb von Funktionen
 - wende gewisse Regeln an, Fokus auf dominanten Termen

Additionsgesetz für $O()$:

- wird oftmals für sequentiell Verhaltensweisen angewendet
- $O(f(n)) + O(g(n)) = O(f(n) + g(n))$
- Zum Beispiel:

```
for i in range(n):  
    for j in range(n):  
        print("b")
```

**n Schleifendurchläufe
je $O(n)$**

$O(n)$

$\rightarrow O(n^2)$

ist **$O(n^2)$** da es **n** äußere Schleifendurchläufe gibt und jeder innere Schleifendurchlauf **n** Schritte benötigt.

Typische Laufzeitasymptotiken von Algorithmen

- $O(1)$ bezeichnet **konstantes Wachstum**
- $O(\log n)$ bezeichnet **logarithmisches Wachstum**
- $O(n)$ bezeichnet **lineares Wachstum**
- $O(n \log n)$ bezeichnet **log-lineares Wachstum**
- $O(n^2)$ bezeichnet **quadratisches Wachstum**
- $O(n^3)$ bezeichnet **kubisches Wachstum**
- **polynomielles Wachstum bedeutet** $O(n^c)$ für eine Konstante $c \geq 1$
- **Exponentielles Wachstum** $O(c^n)$ bedeutet für eine Konstante $c > 1$



aufsteigend sortiert

Asymptotik anschaulicher

Wachstum	n=10	n=100	n=1000	n=1000000
$O(1)$	1	1	1	1
$O(\log n)$	4	7	10	16
$O(n)$	10	100	1000	1000000
$O(n \log n)$	40	700	10000	160000000
$O(n^2)$	100	10000	10000000	1000000000000
$O(2^n)$	1024	1267650600 2282294014 9670320537 6	1071508607186267320948425 0490600018105614048117055 3360744375038837035105112 4936122493198378815695858 1275946729175531468251871 4528569231404359845775746 9857480393456777482423098 5421074605062371141877954 1821530464749835819412673 9876755916554394607706291 4571196477686542167660429 8316526243868372056680693 76	Viel Glück! (hat 301030 Stellen)

Lineare Laufzeit

```
def lineare_suche(L, e):  
    gefunden = False  
    for i in range(len(L)):  
        if e==L[i]:  
            gefunden = True  
    return gefunden
```

*hier könnten wir die Laufzeit verbessern, indem wir direkt **True** zurückgeben, aber dies würde den Worst Case nicht wirklich umgehen können*

- notfalls muss die komplette Liste durchlaufen werden, um festzustellen, dass ein Element nicht vorhanden ist
- $O(\text{len}(L))$ Schleifendurchläufe * $O(1)$ um $e==L[i]$ zu testen
 - $O(1+4n+1)=O(4n+2)=O(n)$
- Gesamtlaufzeit ist **$O(n)$** , wenn **n die Länge $\text{len}(L)$** bezeichnet

hier nehmen wir natürlich, an dass wir in konstanter Zeit auf ein Listenelement zugreifen können

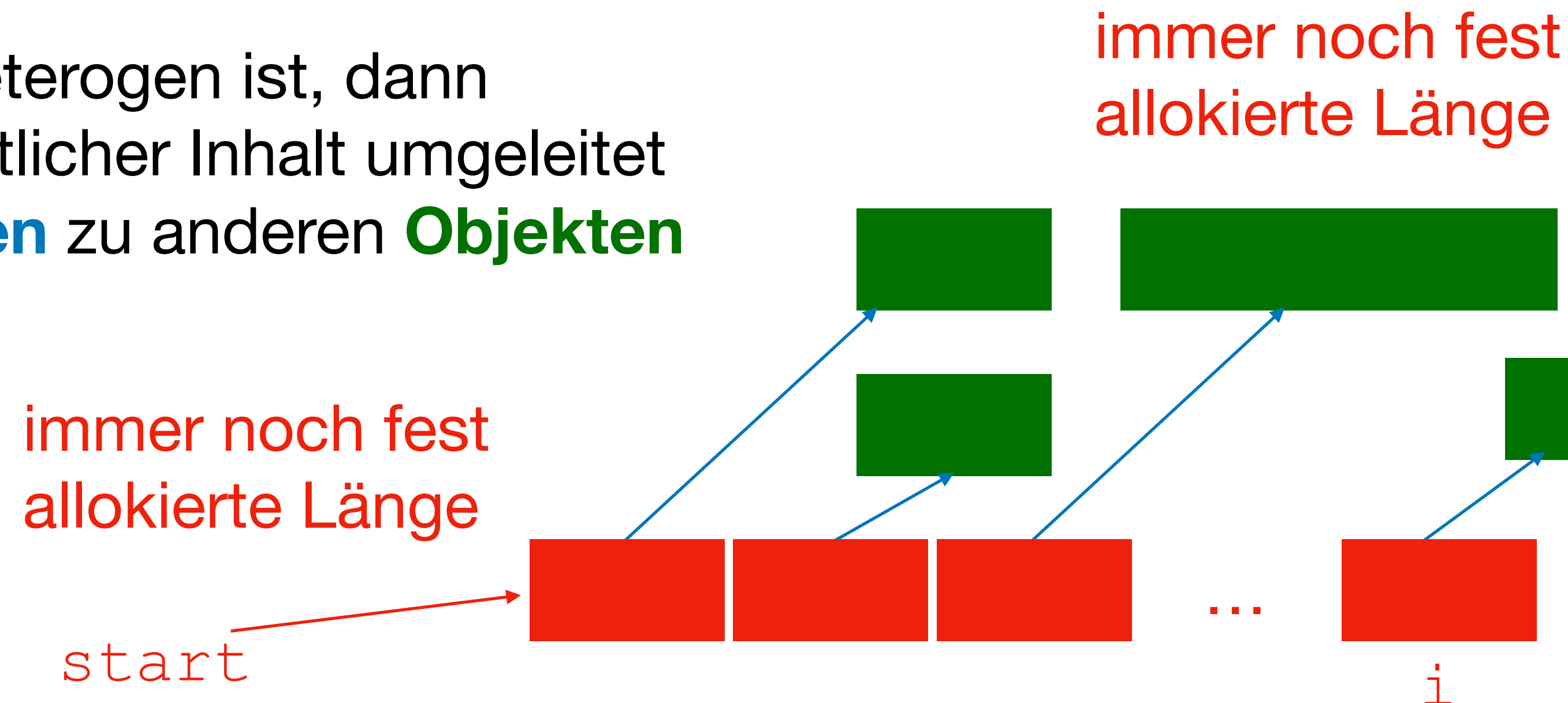
Listenzugriff in $O(1)$

- wenn Liste **nur aus ints** besteht, dann ist das i -te Element an Position $\text{start} + 4 * i$



fest allokierte Länge,
(bspw. 4 Bytes)

- falls Liste heterogen ist, dann
 - wird eigentlicher Inhalt umgeleitet
 - Referenzen** zu anderen **Objekten**



Lineare Laufzeit auf unsortierten Listen

```
def komische_suche(L, e):  
    for i in range(len(L)):  
        if e==L[i]:  
            return True  
        if L[i]>e:  
            return False  
    return False
```

im Worst Case muss die Liste immer noch komplett durchlaufen werden

- es muss lediglich gewartet werden, bis eine Zahl größer e erreicht wird (um abubrechen) oder davor abzulehnen
- $O(\text{len}(L))$ für die Schleife * $O(1)$ für den Test $e==L[i]$
- **Gesamtlaufzeit ist $O(n)$** wenn n die Länge $\text{len}(L)$ bezeichnet
- **Man beachte:** Die Größenordnung der Laufzeit dieses Algorithmus für den Worst Case ist dieselbe wie die des vorherigen Algorithmus

Lineare Laufzeit

wenn wir nichts weiteres über diese Liste wissen (z.B. dass sie sortiert vorliegt)

- ist im Allgemeinen notwendig, um ein Element in einer Liste zu finden
- Beispiel: Vertausche eine Zeichenkette

```
def spiegeln(s):  
    ergebnis=""  
    for c in s:  
        ergebnis = c + ergebnis  
    return ergebnis
```

- Laufzeit($O(\text{len}(s))$)

Lineare Laufzeit

- Komplexität hängt oftmals von der Anzahl der Iterationen ab

```
def fakultaet_iterativ(n):  
    prod = 1  
    for i in range(1, n+1):  
        prod *= i  
    return prod
```

- Anzahl der Schleifendurchläufe ist n
- Anzahl der Operationen innerhalb einer Schleife ist konstant (hier 4: inkrementiere i, weise Summe i zu, multipliziere, weise Produkt zu)
 - $O(1+4n+1)=O(4n+2)=O(n)$
- insgesamt nur **$O(n)$**

Verschachtelte Schleifen

aber nicht immer!

- **einfache Schleifen** bedeuten oftmals **lineare Laufzeit**
- wie sieht es mit **verschachtelten Schleifen** (d.h. Schleifen in Schleifen) aus?
- Generelle Antwort: Kann manchmal schwierig aber oftmals auch einfach zu analysieren sein

Quadratische Komplexität

*Listen sind keine Mengen sondern
Sequenzen!*

- **Beispiel:** Bestimme ob die erste Liste „in der zweiten enthalten ist“, genauer ob jedes Element der ersten Liste auch in der zweiten Liste irgendwo vorkommt (ohne Vielfachheiten zu zählen)

```
def istEnthalten(L1,L2):  
    for e1 in L1:  
        gefunden = False  
        for e2 in L2:  
            if e1 == e2:  
                gefunden = True  
                break  
        if not gefunden:  
            return False  
    return True
```


Quadratische Komplexität

```
def istEnthalten(L1,L2):  
    for e1 in L1:  
        gefunden = False  
        for e2 in L2:  
            if e1 == e2:  
                gefunden = True  
                break  
        if not gefunden:  
            return False  
    return True
```

- äußere Schleife wird **len**(L1) Mal durchlaufen
- jede Iteration (jeder Schleifendurchlauf) führt **len**(L2) innere Iterationen durch, die wiederum jeweils konstant viele Operationen durchführen

$O(\text{len}(L1) * \text{len}(L2))$

- Worst Case tritt bspw. ein, wenn alle Elemente in L1 gleich einem Element sind und dieses Element in L2 nur an letzter Position auftaucht

Quadratische Komplexität

*Listen sind keine Mengen sondern
Sequenzen!*

- **Beispiel:** Bestimme den „Durchschnitt“ zweier Listen, genauer eine Sequenz der Elemente, die sowohl mindestens einmal in beiden Listen vorkommen ohne Vielfachheiten (d.h. die Ergebnisliste darf jedes Element nur einmal enthalten)

```
def schneide(L1,L2):  
    tmp = []  
    for e1 in L1:  
        for e2 in L2:  
            if e1 == e2:  
                tmp.append(e1)  
                break  
    ergebnis = []  
    for e in tmp:  
        if not (e in ergebnis):  
            ergebnis.append(e)  
    return ergebnis
```

Quadratische Komplexität

```
def schneide(L1, L2):  
    tmp = []  
    for e1 in L1:  
        for e2 in L2:  
            if e1 == e2:  
                tmp.append(e1)  
                break  
    ergebnis = []  
    for e in tmp:  
        if not (e in ergebnis):  
            ergebnis.append(e)  
    return ergebnis
```

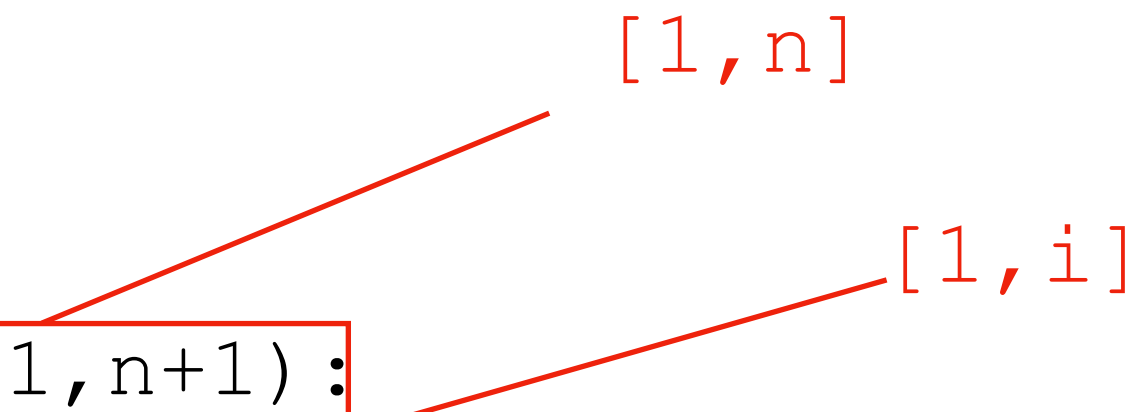
Ist $O(\text{len}(L1))$

wird $O(\text{len}(L1))$ mal durchlaufen

- erste (geschachtelte) Schleife hat eine Gesamtlaufzeit von $O(\text{len}(L1) * \text{len}(L2))$
 - zweite Schleife hat Laufzeit $O(\text{len}(L1)^2)$
- $O(\text{len}(L1) * \max(\text{len}(L1), \text{len}(L2)))$
- Worst Case tritt bspw. ein, wenn es ein Element e gibt so, dass $L1$ die Form $[e, \dots, e]$ hat und $L2$ die Form $[e]$ hat oder auch wenn $L1$ und $L2$ gleiche Länge haben und disjunkt sind.

Laufzeit geschachtelter Schleifen

```
def g(n):  
    x = 0  
    for i in range(1, n+1):  
        for j in range(1, i+1):  
            x += 1  
    return x
```

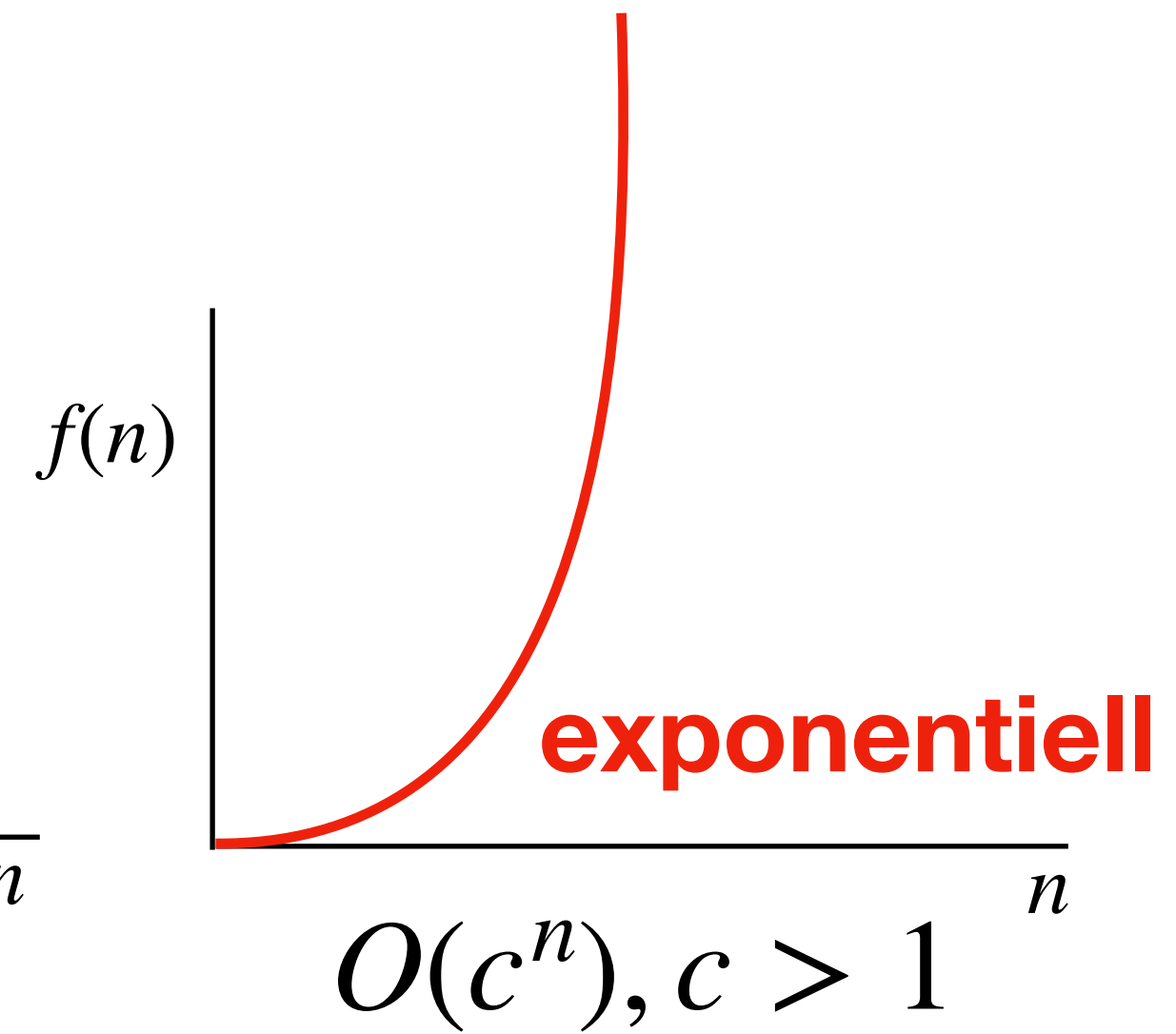
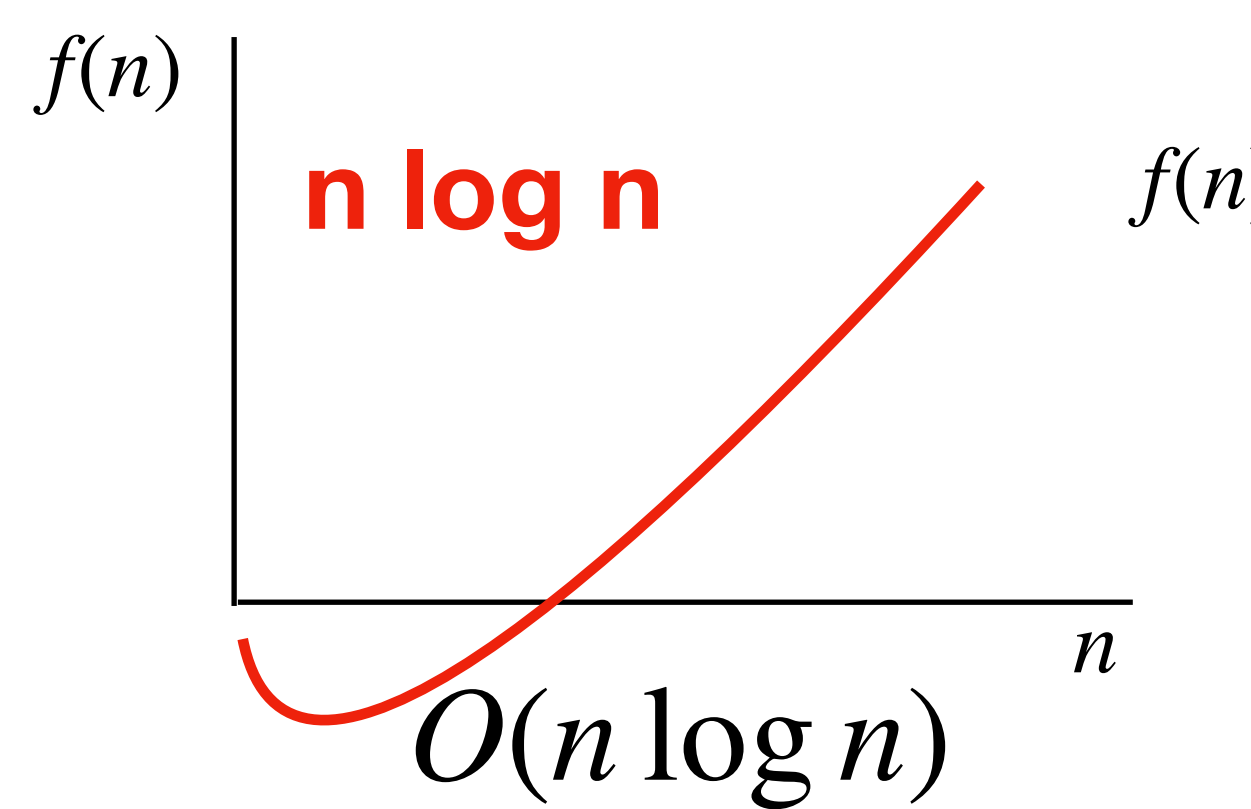
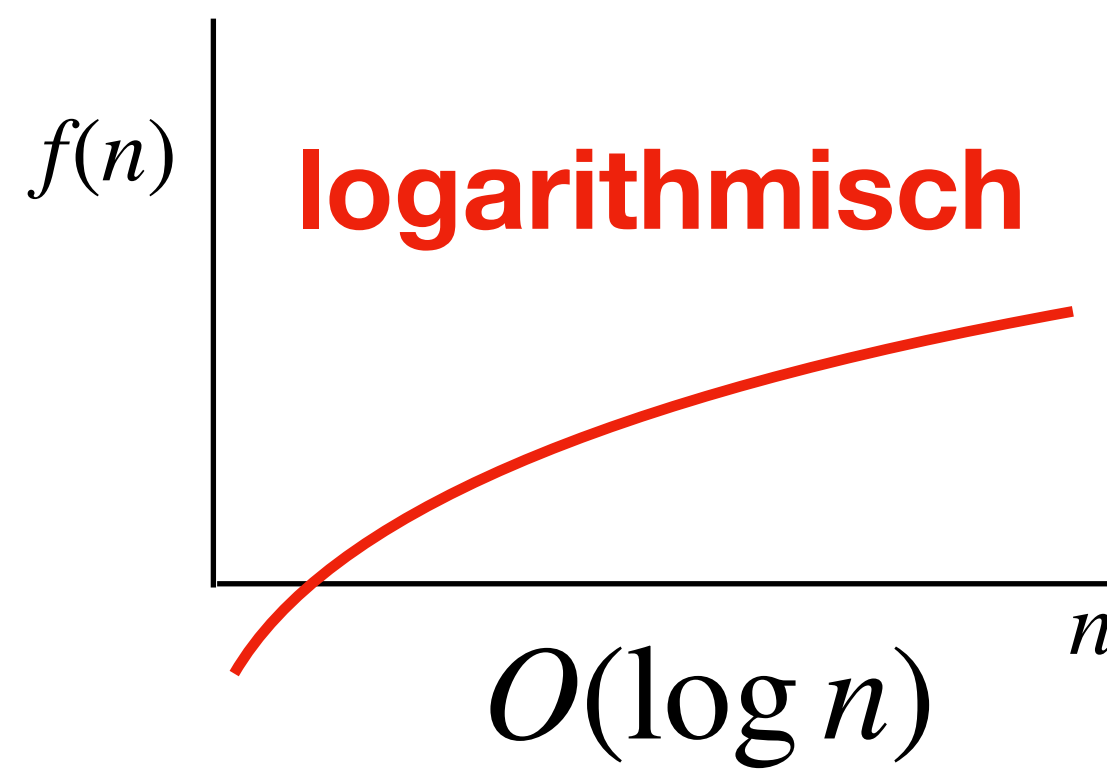
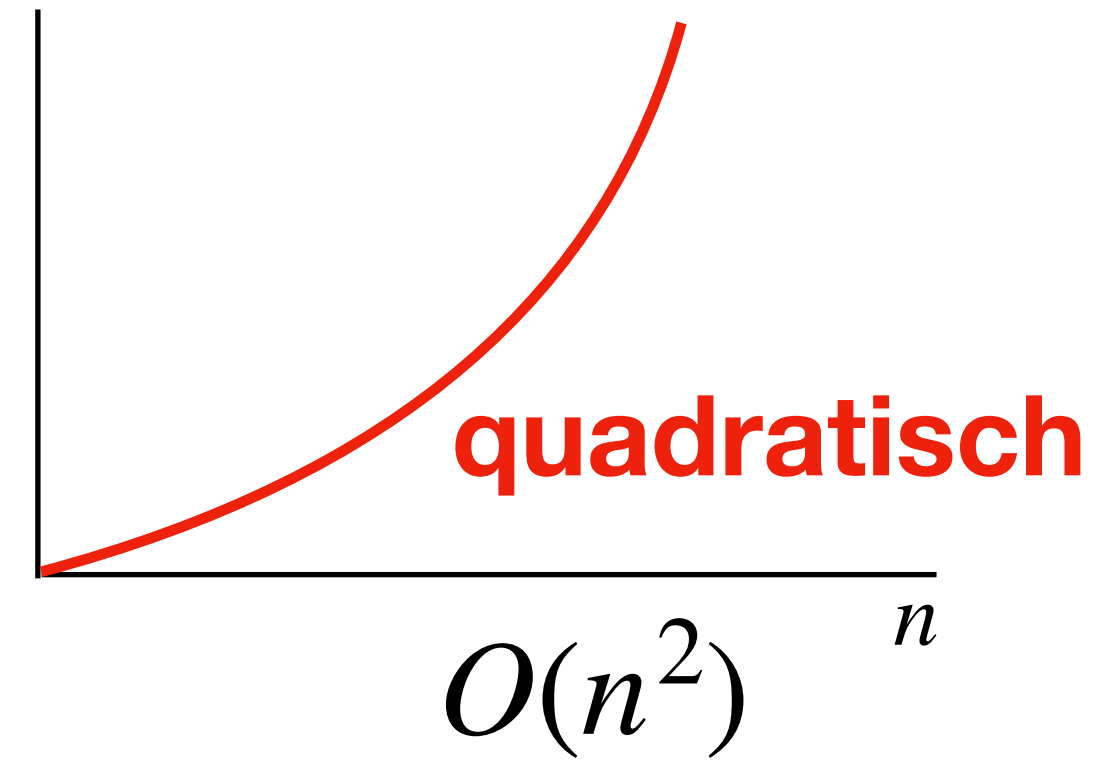
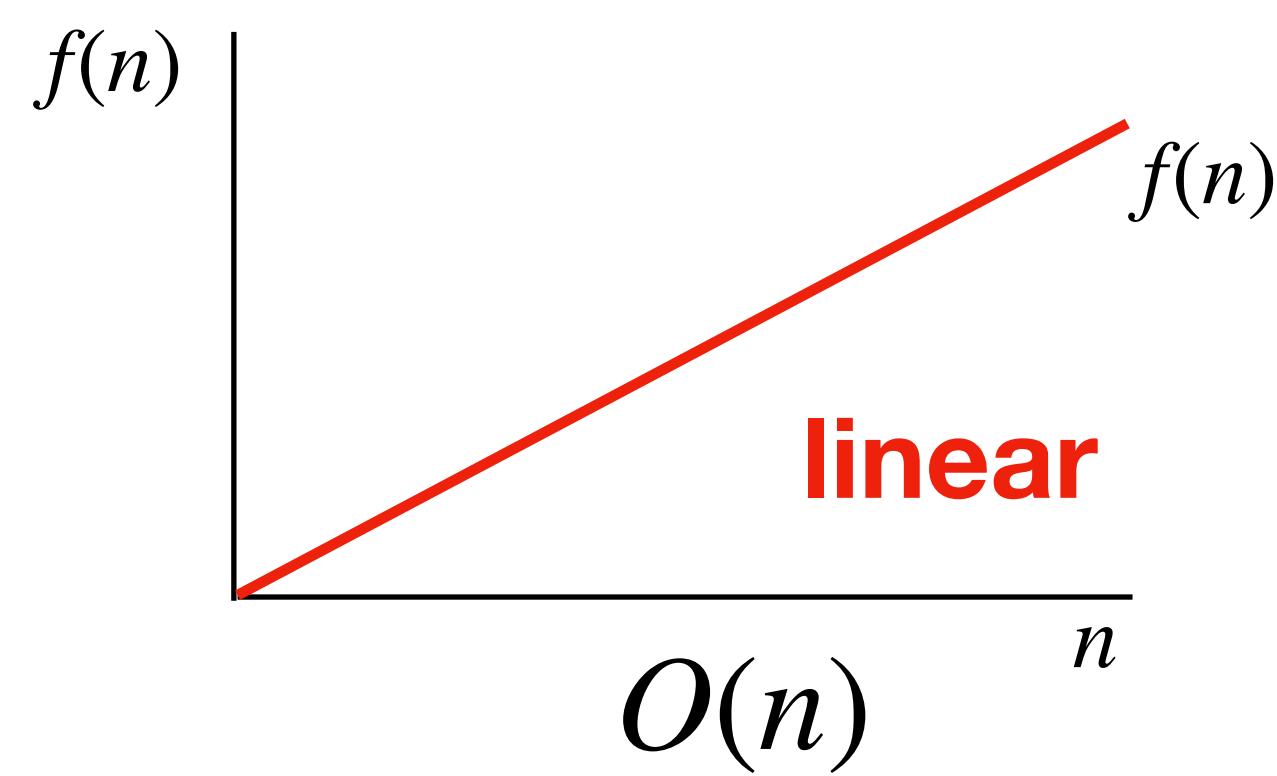
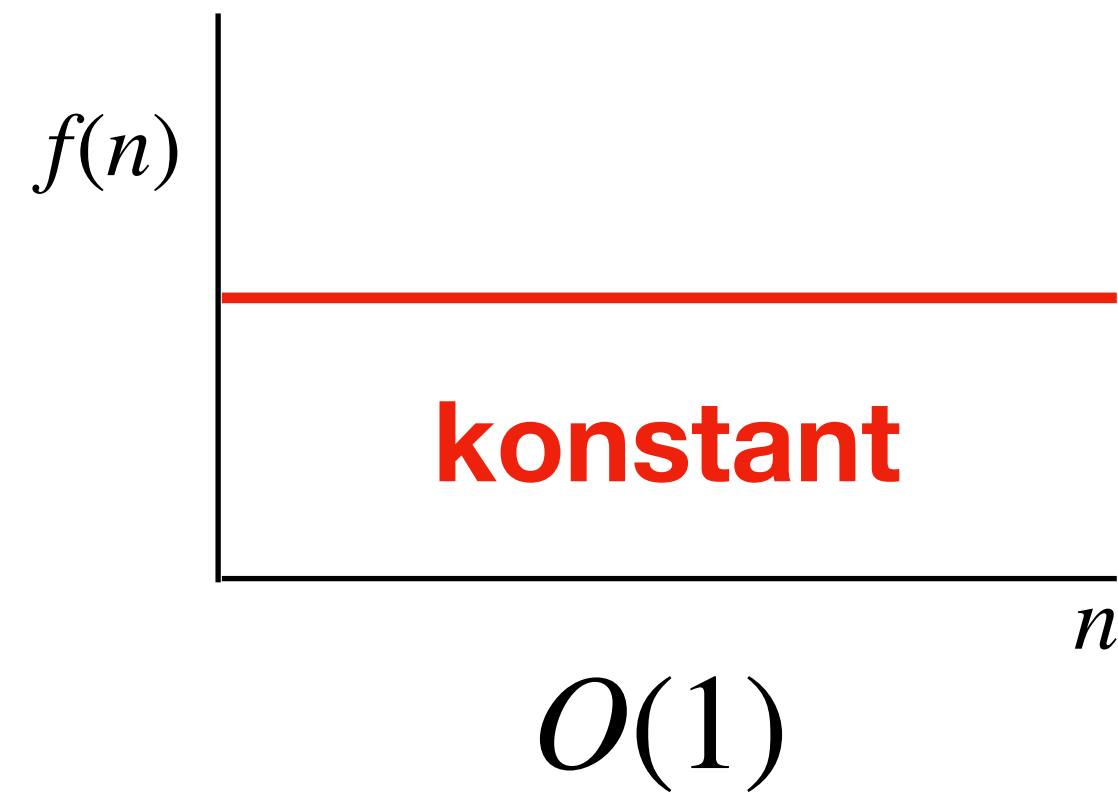


- was berechnet dieses Programm?
- es berechnet $1 + 2 + 3 + \dots + n$
- es gibt n äußere Schleifendurchläufe
- beim i -ten äußeren Schleifendurchlauf gibt es i innere Schleifendurchläufe
- Erinnerung:

$$\sum_{i=1}^n i = n \cdot (n + 1) / 2$$

- **Gesamtlaufzeit: $O(n^2)$**

Arten von Größenwachstum



Typische Laufzeitasymptotiken von Algorithmen

- $O(1)$ bezeichnet **konstantes Wachstum**
- $O(\log n)$ bezeichnet **logarithmisches Wachstum**
- $O(n)$ bezeichnet **lineares Wachstum**
- $O(n \log n)$ bezeichnet **log-lineares Wachstum**
- $O(n^2)$ bezeichnet **quadratisches Wachstum**
- $O(n^3)$ bezeichnet **kubisches Wachstum**
- **polynomielles Wachstum** bedeutet $O(n^c)$ für eine Konstante $c \geq 1$
- **Exponentielles Wachstum** $O(c^n)$ bedeutet für eine Konstante $c > 1$



aufsteigend sortiert

Konstante Laufzeit

- bedeutet, dass die Laufzeit unabhängig von der Eingabelänge ist
- insbesondere kann nicht die komplette Eingabe berücksichtigt werden
- Schleifen durchaus auch bei konstanter Laufzeit möglich, wenn die Anzahl der Durchläufe bzw. die Gesamtlaufzeit nicht von der Eingabe abhängt
- es gibt kaum interessante Algorithmen mit konstanter Laufzeit

Logarithmische Laufzeit

- auch bei solchen Algorithmen wird nicht die komplette Eingabe berücksichtigt
- Übungsaufgabe: $\forall c, \forall d \in \mathbb{R}, c > 1, d > 1, O(\log_c(n)) = O(\log_d(n))$
- **Binäre Suche hat logarithmische Laufzeit**, da in jeder Iteration das zu explorierende Intervall **halbiert** wird
- nächstes Beispiel hierfür: Bisection Search

Bisection Search

- nehmen wir an wir wollen ein bestimmtes Element e in einer gegebenen Liste $\mathbb{L}$ suchen
- wir haben bereits einen Algorithmus (Sequenzielle Suche) kennengelernt, der die komplette Liste durchläuft, dieser hat eine Laufzeit $O(n)$, wobei n die Länge der Liste beschreibt.
- nehmen wir nun an, dass die Liste aufsteigend sortiert ist
 - wir können nach wie vor sequentiell suchen, $O(n)$
 - können wir ein (laufzeit-)effizienteres Verfahren angeben?

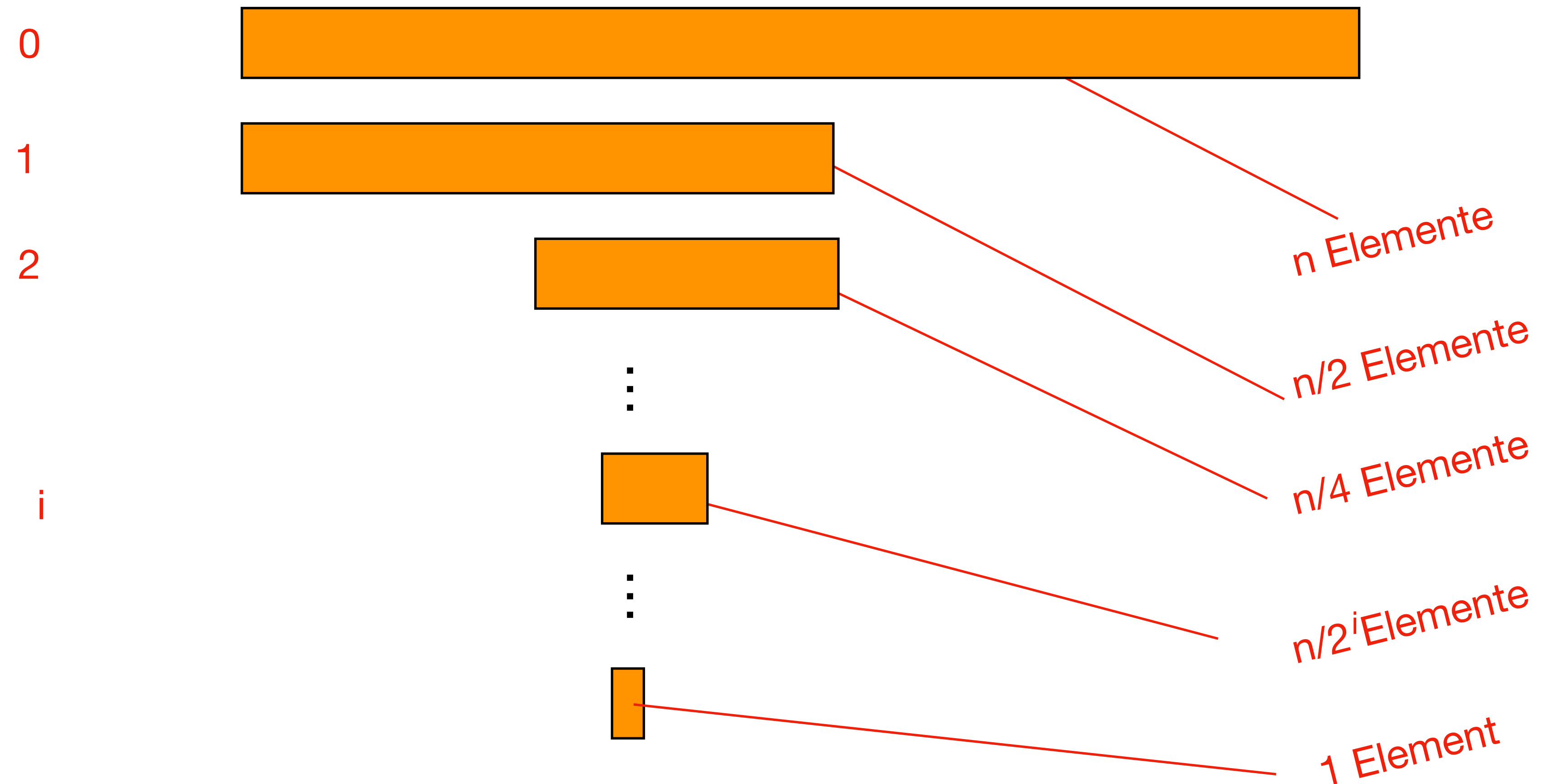
Bisection Search, Idee

1. wähle einen Index i , der die Liste in zwei Teile teilt
2. teste ob $L[i] == e$
3. falls ungleich, teste ob $L[i] < e$ oder ob $L[i] > e$
4. abhängig vom Ergebnis wird entweder in der linken oder der rechten Hälfte gesucht

Dies entspricht dem **Divide-and-Conquer Algorithmus**

- teile Problem in kleinere Teilprobleme (hier: kleinere Liste) plus ein paar elementare Operationen (hier: Vergleiche)
- Falls noch nicht fertig, fahre mit kleinerem Problem fort

Analyse von Bisection Search



- wir sind erst fertig, wenn das erste Mal gilt

$$1 \geq n/2^i \implies 2^i \geq n \implies i \geq \log(n)$$

- Gesamtlaufzeit **$O(\log(n))$**

Bisection Search, Algorithmus 1

```
def bisection1(L, e):
```

```
    if L == []:  
        return False
```

konstant
 $O(1)$

```
    elif len(L) == 1:  
        return L[0] == e
```

konstant
 $O(1)$

```
    else:
```

```
        haelfte = len(L) // 2
```

konstant
 $O(1)$

```
        if L[haelfte] > e:  
            return bisection1(L[:haelfte], e)
```

```
        else:  
            return bisection1(L[haelfte:], e)
```

Liste wird kopiert!

nicht konstant
 $O(\text{len}(L))$

nicht konstant
 $O(\text{len}(L))$

Laufzeit-Analyse Algorithmus 1

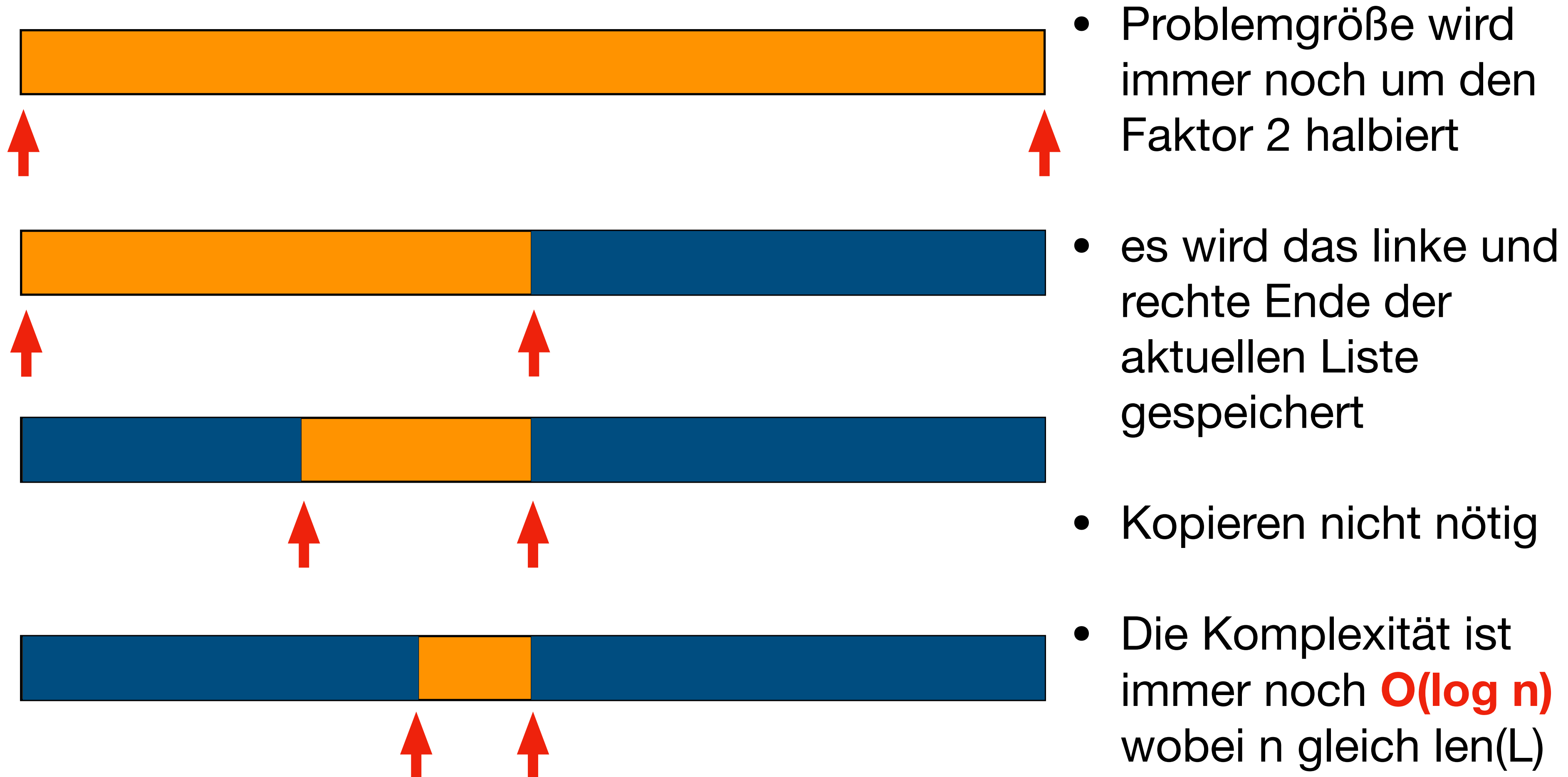
- $O(\log n)$ Rekursionsaufrufe
 - in jedem Rekursionsaufruf wird Listenlänge halbiert
 - wenn die ursprüngliche Liste Länge n hat, dann ist im Worst Case die Listenlänge irgendwann 1, also wenn $n/2^k \leq 1$
- $O(n)$ innerhalb des ersten Funktionsaufrufs, um die halbe Liste (die linke oder rechte Hälfte) zu kopieren
 - genauer $n/2$ für den ersten Aufruf, $n/4$ für den zweiten, ...
 $n/2^k$ für den k -ten
- Worst-Case Analyse: $O(\log n) \cdot O(n) = O(n \log n)$

- Genauere Analyse:

$$\sum_{k=1}^{\infty} n/2^k \leq n \quad \rightarrow \quad O(n)$$

also nicht wirklich eine Verbesserung zur sequentiellen Suche

Analyse von Bisection Search



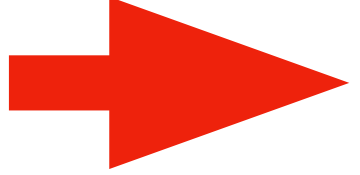
Bisection Search Algorithmus 2

```
def bisection2(L,e):  
    def bisection_hilfe(L, e, links, rechts):  
        if links >= rechts:  
            return links==rechts and L[links] == e  
        mitte = (links + rechts) // 2  
        if L[mitte] == e:  
            return True  
        elif L[mitte] > e:  
            return bisection_hilfe(L,e,links,mitte-1)  
        else:  
            return bisection_hilfe(L,e,mitte+1,rechts)  
    if len(L) == 0:  
        return False  
    else:  
        return bisection_hilfe(L,e,0,len(L)-1)
```

*konstante statt
lineare Laufzeit*

*konstante statt
lineare Laufzeit*

Laufzeit-Analyse Algorithmus 2

- $O(\log n)$ Rekursionsaufrufe
 - in jedem Rekursionsaufruf wird Listenlänge halbiert
 - wenn die ursprüngliche Liste Länge n hat, dann ist im Worst Case die Listenlänge irgendwann 1, also wenn $n/2^k \leq 1$
- linker und rechter Index (und ursprüngliche Liste und gesuchtes Element) werden übergeben
- Liste wird nicht kopiert, nur linker und rechter Index werden ggf. neu gesetzt
- innerhalb eines Funktionsaufrufs also Laufzeit $O(1)$
- Naive Analyse: $O(\log n) * O(1)$  **$O(\log n)$**

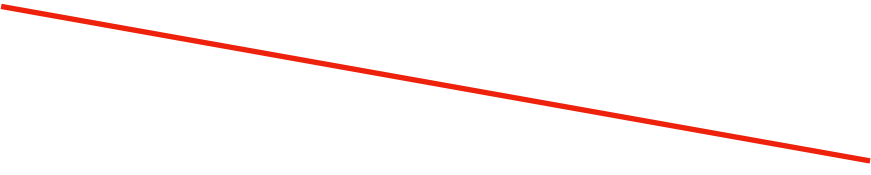
Rekursionstiefe

Logarithmische Laufzeit

```
def intnachstr(n):  
    ziffern = '0123456789'  
    if n==0:  
        return '0'  
    else:  
        return intnachstr(n//10)+ziffern[n%10]
```

Was macht folgendes Programm?

Es wandelt eine Zahl in einen String zur Basis 10 um.



inklusive führender
Nullen

Logarithmische Laufzeit

```
def intnachstr(n):  
    ziffern = '0123456789'  
    if n==0:  
        return '0'  
    else:  
        return intnachstr(n//10)+ziffern[n%10]
```

- Konstante Laufzeit innerhalb eines Funktionsaufrufs
- Bei rekursivem Aufruf wird die Eingabe verkleinert, nämlich durch 10 geteilt (Integer-Division)
- Rekursionstiefe entspricht die Anzahl der Male, die man n durch 10 teilen kann, um schließlich 0 zu werden.
- **$O(\log n)$**

Nochmal lineare Laufzeit

```
def fact_iter(n):  
    prod = 1  
    for i in range(1, n+1):  
        prod *= i  
    return prod
```

- insgesamt **$O(n)$** Schleifendurchläufe (genau n), konstante Zeit pro Schleifendurchlauf

Nochmal lineare Laufzeit

```
def fakultaet(n):  
    if n==1:  
        return 1  
    else:  
        return n*fakultaet(n-1)
```

- berechnet die Fakultätsfunktion rekursiv
- genau genommen (bei genauem Messen mittels Stoppuhr) kann man feststellen, dass die mittels Stoppuhr gemessene Zeit langsamer als die iterative Variante ist
- jedoch immer noch **$O(n)$** , denn die Rekursionstiefe ist linear in n , mit konstanter Laufzeit innerhalb eines Funktionsaufrufs
- Iterative und rekursive Implementierungen für die Fakultätsfunktion haben dieselbe asymptotische Laufzeit, nämlich $O(n)$

Log-lineare Laufzeit

- viele praktische Algorithmen haben log-lineare Laufzeit, also $O(n \log n)$
- häufig verwendete log-lineare Algorithmus ist Merge-Sort (später mehr)
- ist die inhärente^{???} Laufzeit für Sortieren (später mehr)

Polynomielle Laufzeit

- viele nicht-lineare Algorithmen haben quadratische Laufzeit
- entsteht häufig, wenn wir geschachtelte Schleifen haben
- haben wir **bisher** kennengelernt
- entspricht im Wesentlichen dem, was man als „effizient berechenbar“ bezeichnet (superpolynomielle bezeichnet man typischerweise als ineffizient)

Exponentielle Laufzeit

- Die meisten rekursiven Algorithmen besaßen nur einen rekursiven Aufruf. (keine streng mathematische Aussage, mit Vorsicht zu genießen!)
- Bei den Türmen von Hanoi und der rekursiven Berechnung von `fib` (ohne Wörterbuch) war das anders: Zwei rekursive Aufrufe

```
def Tuerme(n, von, nach, zwischen):
```

```
    if n==1:
```

```
        print(von+ " -> " + nach)
```

```
    else:
```

```
        Tuerme(n-1, von, zwischen, nach)
```

```
        Tuerme(1, von, nach, zwischen)
```

```
        Tuerme(n-1, zwischen, nach, von)
```

```
def fib(n):
```

```
    if n==0 or n==1:
```

```
        return 1
```

```
    else:
```

```
        return fib(n-1) + fib(n-2)
```

(Genaue Laufzeit werden wir noch exakt bestimmen)

- Einige Probleme sind vermutlich inhärent exponentiell, bei den meisten von ihnen ist das aber tatsächlich noch nicht bekannt!!!
 - sehr schade, da man sehr lange auf Ergebnisse warten muss
 - approximative Lösungen liefern uns oftmals trotzdem zufriedenstellende (aber nicht exakte) Lösungen mit denen wir etwas anfangen können

Laufzeit von Türme von Hanoi

- Bezeichne t_n die Laufzeit, um Türme der Höhe n zu berechnen
- Es gilt folgende Rekurrenz $t_1 = 1$ und $t_n = 2t_{n-1} + 1$ für alle $n > 1$

$$\begin{aligned}\text{also } t_n &= 2t_{n-1} + 1 \\ &= 2(2t_{n-2} + 1) + 1 \\ &= 4t_{n-2} + 2 + 1 \\ &= 4(2t_{n-3} + 1) + 2 + 1 \\ &= 2^{n-1} + 2^{n-2} + \dots + 4 + 2 + 1 \\ &= 2^n - 1\end{aligned}$$

- die Gesamtlaufzeit ist also **$O(2^n)$**

Exponentielle Laufzeit

- gegeben sei eine Menge von Integers (d.h. ohne Wiederholungen) gegeben als Liste
- wir wollen die Menge aller Teilmengen (Untermengen) berechnen
- bspw. besitzt die Menge $\{1,2,3,4\}$ folgende Untermengen:
 - $\{\}, \{1\}, \{2\}, \{3\}, \{4\}, \{1,2\}, \{1,3\}, \{1,4\}, \{2,3\}, \{2,4\}, \{3,4\},$
 $\{1,2,3\}, \{1,2,4\}, \{1,3,4\}, \{2,3,4\}, \{1,2,3,4\}$
- man beachte, dass es nicht auf die Ordnung der Elemente ankommt also bspw. $\{1,2,3,4\}=\{3,2,1,4\}$ oder $\{1,2\}=\{2,1\}$

Potenzmengenkonstruktion, Idee

- wie wollen die Potenzmenge (Menge aller Untermengen) von $\{1, \dots, n\}$ generieren
- nehmen wir an wir könnten bereits die Potenzmenge von $\{1, \dots, n-1\}$ generieren
- dann ist die Potenzmenge von $\{1, \dots, n\}$ die Menge aller Mengen, die entsteht, wenn für jede Menge X in der Potenzmenge von $\{1, \dots, n-1\}$ sowohl X als auch $X \cup \{n\}$ aufnehmen
 - $\{\{\}, \{1\}, \{2\}, \{1,2\}, \{3\}, \{1,3\}, \{2,3\}, \{1,2,3\}, \{4\}, \{1,4\}, \{2,4\}, \{1,2,4\}, \{3,4\}, \{1,3,4\}, \{2,3,4\}, \{1,2,3,4\}\}$
- liefert ein einfaches rekursives Verfahren

Potenzmengenkonstruktion

Algorithmus

```
def generiereUntermenge(L):
```

```
    ergebnis = []
```

```
    if len(L) == 0:
```

```
        return [[]]
```

```
    kleiner = generiereUntermenge(L[:-1])
```

```
    extra = L[-1:]
```

```
    neu = []
```

```
    for klein in kleiner:
```

```
        neu.append(klein+extra)
```

```
    return kleiner + neu
```

Liste, die nur eine
leere Liste enthält

Eingabeliste ohne das letzte Element

Einelementige Liste, die nur das letzte Element enthält

Potenzmengenkonstruktion

Laufzeitanalyse

```
def generiereUntermenge(L):  
    ergebnis = []  
    if len(L) == 0:  
        return [[]]  
    kleiner = generiereUntermenge(L[:-1])  
    extra = L[-1:]  
    neu = []  
    for klein in kleiner:  
        neu.append(klein+extra)  
    return kleiner + neu
```

wir nehmen an append
hat Laufzeit $O(1)$

wesentliche Gesamtlaufzeit:

Laufzeit für das kleinere Problem plus die Laufzeit, um Lösung des kleineren Problems einmal zu duplizieren.

Potenzmengenkonstruktion

Laufzeitanalyse

- bezeichne t_n die Laufzeit für Eingabegröße n
- bezeichne s_n die Größe der Lösung für Eingabegröße n
- es gilt folgende Rekurrenz: $t_n \leq t_{n-1} + c \cdot (s_{n-1} + n)$
für eine Konstante c

- es gilt $t_n \leq t_{n-1} + c \cdot (s_{n-1} + n)$
 $= t_{n-1} + c \cdot (2^{n-1} + n)$

$$s_0 = 1 \text{ und } s_n = 2s_{n-1} \text{ für alle } n > 0$$

$$\text{also } s_n = 2^n \text{ für alle } n \geq 0$$

$$\leq t_{n-2} + c \cdot (s_{n-2} + n - 1) + c \cdot (2^{n-1} + n)$$

$$\leq t_{n-k} + c \cdot \left(2^{n-k} + \dots + 2^{n-1} + 2 \cdot \sum_{i=n-k+1}^n i \right)$$

$$= \boxed{t_0} + c \cdot (2^0 + \dots + 2^{n-1} + n(n+1))$$

2

$$= 2 + c \cdot (2^n - 1 + n(n+1))$$

liegt in $O(2^n)$

Typische Algorithmen für typische Laufzeiten

- $O(1)$ - Programm hängt nicht von der Größe des Problems ab
- $O(\log n)$ - Verkleinere Eingabe wiederholt durch einen konstanten Faktor größer 1
- $O(n)$ - Einfache iterative oder rekursive Programme
- $O(n \log n)$ - sehen wir gleich
- $O(n^c)$ - mehrere geschachtelte Schleifen oder rekursive Aufrufe
- $O(c^n)$ - mehrere Rekursionsaufrufe in jeder Stufe

Wieder keine streng mathematische Aussage, eher eine Tendenz!
Es gibt einige Gegenbeispiele!

Fibonaccizahlen iterativ

```
def fib_iter(n):
```

```
    if n == 0:
```

```
        return 1
```

```
    elif n == 1:
```

```
        return 1
```

```
    else:
```

```
        fib_i = 1
```

```
        fib_ii = 1
```

```
        for i in range(n-1):
```

```
            tmp = fib_i
```

```
            fib_i = fib_ii
```

```
            fib_ii = tmp + fib_i
```

```
        return fib_ii
```

konstant
 $O(1)$

konstant
 $O(1)$

linear
 $O(n)$

konstant
 $O(1)$

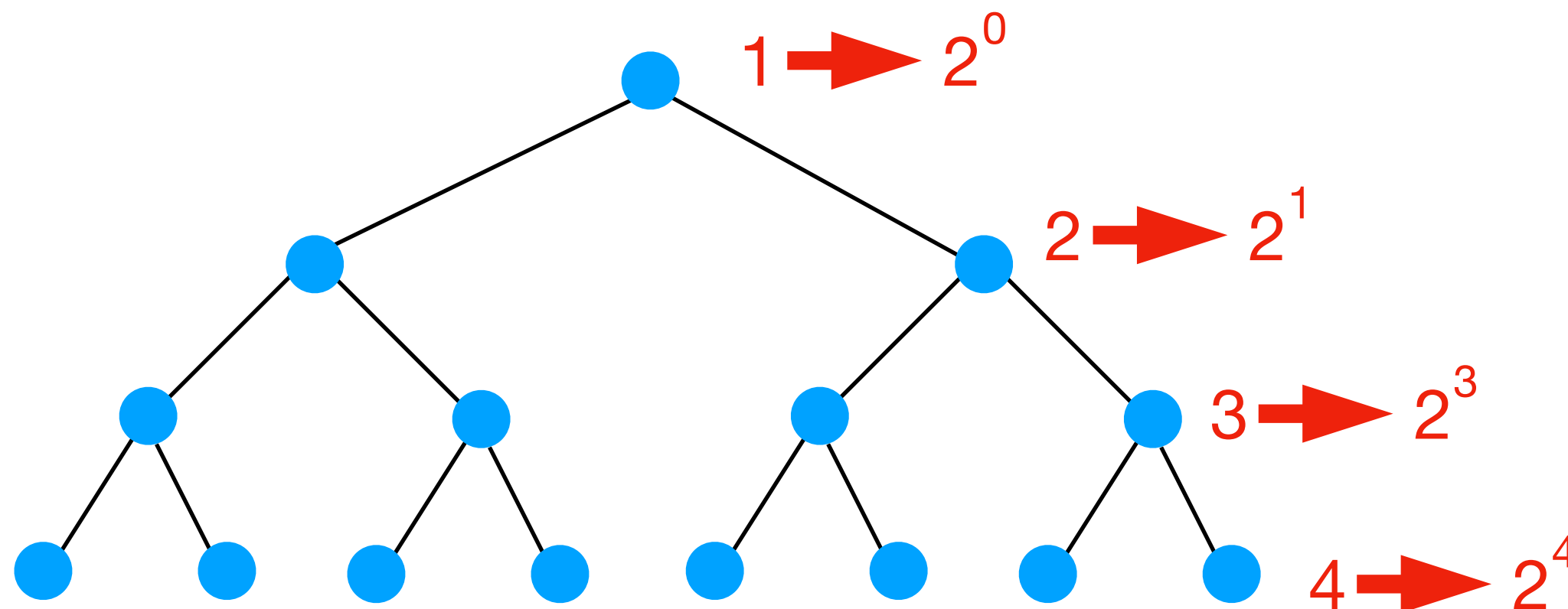
- Worst Case:
 $O(n)$

Laufzeit von Fibonacci rekursiv

(ohne Wörterbuch)

```
def fib(n):  
    if n==0 or n==1:  
        return 1  
    else:  
        return fib(n-1)+fib(n-2)
```

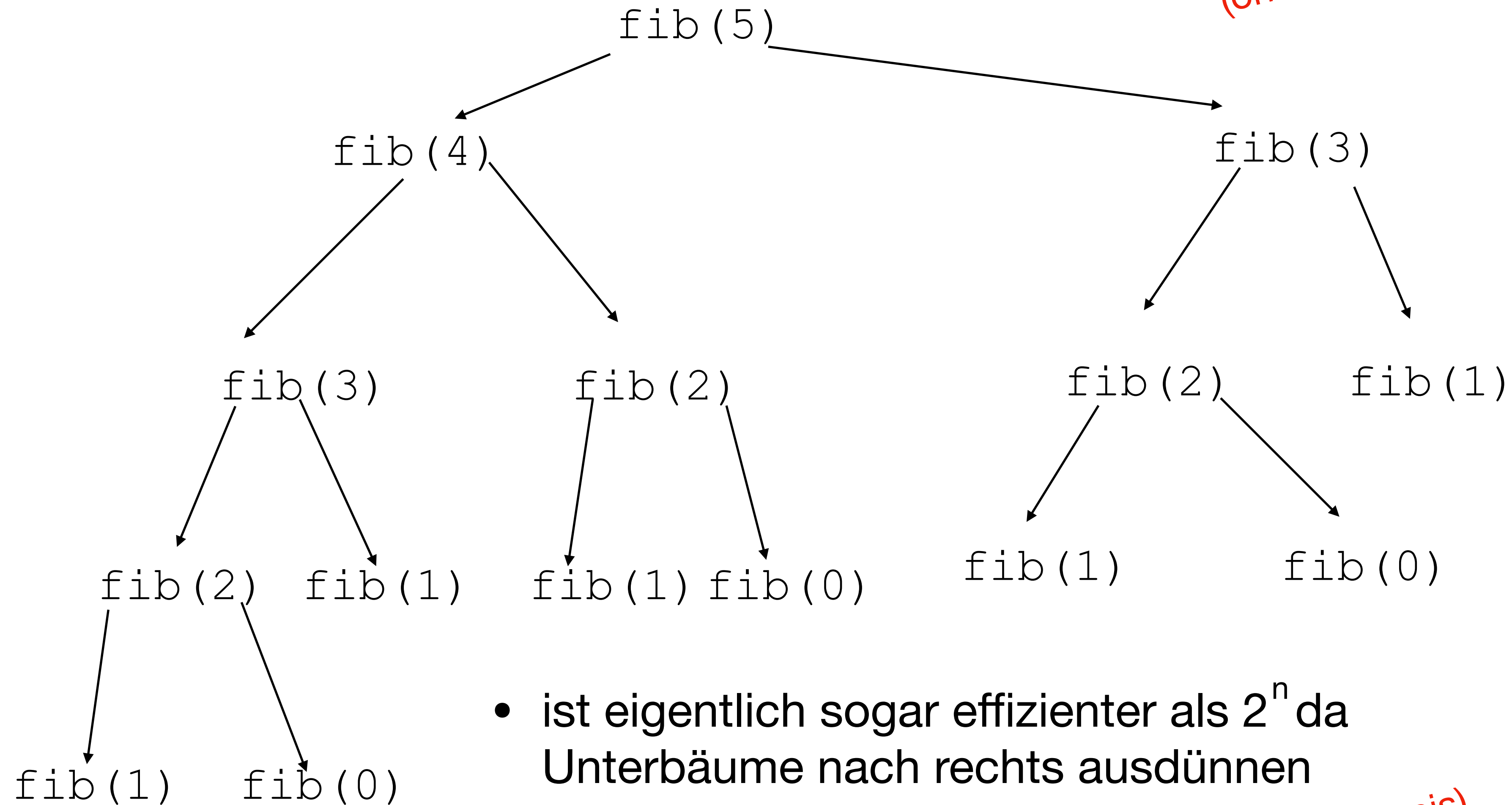
- Worst Case $O(2^n)$ denn $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2) \leq 2 \cdot \text{fib}(n-1)$



entspräche $\text{fib}(n) = 2^n$

Laufzeit von Fibonacci rekursiv

(ohne Wörterbuch)



- ist eigentlich sogar effizienter als 2^n da Unterbäume nach rechts ausdünnen

„Goldener Schnitt“

- es gilt $\lim_{n \rightarrow \infty} \frac{\mathbf{fib}(n+1)}{\mathbf{fib}(n)} = \frac{1 + \sqrt{5}}{2} \approx 1,61803$ (ohne Beweis)

Zusammenfassung O-Notation

- dient die Effizienz eines Algorithmus zu beschreiben
 - ist eine Notation, die das Wachstum beschreibt
 - je kleiner die Größenordnung, desto effizienter
 - maschinen- und implementierungsunabhängig
- Verwenden Sie die O-Notation
 - um die Größenordnung der Laufzeit zu klassifizieren
 - als eine asymptotische Notation
 - als eine obere Schranke
 - bei Worst Case Analysen (meistens dort anzutreffen)

Laufzeit einiger Python-Funktionen

- **Listen** L , n sei $\text{len}(L)$

Worst Case

- | | |
|--|--------|
| • Zugriff ($L[i]$) | $O(1)$ |
| • Speichern (Zuweisung) | $O(1)$ |
| • Länge bestimmen (len) | $O(1)$ |
| • Am Ende anhängen (append) | $O(1)$ |
| • == | $O(n)$ |
| • Element entfernen (remove) | $O(n)$ |
| • Liste kopieren (copy) | $O(n)$ |
| • Liste invertieren (reverse) | $O(n)$ |
| • Einmal traversieren | $O(n)$ |
| • Enthaltensein eines Elements testen | $O(n)$ |

- **Wörterbücher** w , n sei $\text{len}(L)$

Worst Case

- | | |
|---------------------------------------|--------|
| • Zugriff ($w[\text{element}]$) | $O(n)$ |
| • Speichern (Zuweisung) | $O(n)$ |
| • Länge bestimmen (len) | $O(n)$ |
| • Element entfernen (remove) | $O(n)$ |
| • Einmal traversieren | $O(n)$ |

- | | |
|-----------------------------------|--------|
| • Zugriff ($w[\text{element}]$) | $O(1)$ |
| • Speichern (Zuweisung) | $O(1)$ |
| • Element entfernen | $O(1)$ |
| • Einmal traversieren | $O(1)$ |

Average Case

Suchalgorithmen

- Suchalgorithmen sind Methoden, um ein Element in einer Menge von Elementen mit gewissen Eigenschaften zu finden
- zugrundeliegende Menge kann u.U. nur **implizit gegeben** sein
 - Beispiel: Berechnung der Quadratwurzel, mögliche Ansätze:
 - Brute-Force-mäßig alle möglichen Lösungen aufzählen und jeweils testen
 - Bisection Search
 - Newton-Verfahren
- zugrundeliegende Menge kann **explizit gegeben** sein
 - Beispiel: Ist eine gegebene Matrikelnummer in der Menge aller Erstsemester (bspw. als Wörterbuch gegeben) vorhanden?